

Lebanese University
Faculty Of Technology
-Saida-



Senior Project Report

Real-Time Web Platform for University Timetable Consultation and Notifications

Prepared by: RAMADAN Ahmad

Department: Computer and Communication Networks Engineering

Supervisor:
Dr. BAHLAK Samia

Academic Year:
2024-2025

Date of Submission:
4 July 2025

Acknowledgment

I am grateful for the opportunity to work on this senior project, which has significantly enriched my understanding of web development within the context of computer engineering. This project allowed me to apply my academic knowledge to a real-world problem and strengthen my technical and problem-solving skills.

I would like to express my sincere appreciation to all the instructors at the Faculty of Technology for their continuous support and valuable teachings throughout my academic journey. A special thanks to **Dr. BAHLAK Samia Eng. BAHLAK Mohamed** and **Eng. BAHLAK Rime** for their guidance, encouragement, and technical insight during the development of this project.

I would also like to thank all the professors in the CCNE department for maintaining a high academic and technical standard that prepared us for this achievement and I would also like give a special thank to **Professor AL-HAJJAR Abed-El-Salam**, the head of the department, whose exceptional teaching in programming and web development greatly enriched my university experience.

Abstract:

As part of the third-year curriculum in Computer and Communication Network Engineering (CCNE), the Faculty of Technology requires a senior project that bridges theoretical knowledge with practical implementation. My project, completed during the 2024–2025 academic year, involves the development of a web-based platform that enables students and teachers to consult their university timetables in real time and receive instant notifications regarding any schedule changes.

The system is designed to improve communication and organization within academic institutions by offering a centralized, accessible, and user-friendly interface. It features a real-time schedule view, administrative tools for managing course times and user accounts, and a notification system for alerting users to modifications. The platform was developed using React and TypeScript for the frontend, with a Node.js backend and PostgreSQL as the database.

This project not only enhanced my skills in web development and system design, but also taught me valuable lessons in team collaboration, user-centric design, and backend integration. It marks a major milestone in my academic journey and prepares me for further work in the field of web development and software engineering.

Table of Contents

Contents

Cover Page ---- **1**

Acknowledgment ---- **2**

Abstract ---- **3**

Table of Contents ---- **4**

List of figures ---- **7**

List of Tables ---- **9**

Chapter 1 ---- **10**
Introduction ---- 10

1.1 Project Background ---- 10

1.2 Problem Statement ---- 11

1.3 Project Objectives ---- 11

1.4 Project Scope ---- 11

1.5 Target Users ---- 12

1.6 Agile Development Methodology ---- 12

Iterative and Incremental Development ---- 12

Weekly Meetings and Feedback ---- 13

Solo Agile Simulation ---- 13

Task Planning and Workflow ---- 13

Benefits of Agile Use ---- 14

Chapter 2 ---- **15**
Literature Review / Existing Solutions ---- 15

2.1 Overview of Existing Systems ---- 15

2.2 Limitations in Existing Solutions ---- 17

2.3 Motivation for This Project ---- 18

Chapter 3 ---- **19**
System Analysis and Requirements ---- 19

3.1 Functional Requirements ---- 19

3.2 Non-Functional Requirements ---- 19

3.3 Use Case Diagrams ---- 20

3.4 User Roles and Permissions ---- 20

Chapter 4 ---- **21**
System Design and Architecture ---- 21

4.1 System Overview ---- 21

4.3 Component Diagram ---------- 22
4.4 Technologies and Tools Used: ---------- 22
Chapter 5----------**24**
Database Design---------- 24
5.1 Entity-Relationship Diagram ---------- 24
5.2 Tables and Relationships---------- 24
5.3 Database Schema Explanation ---------- 25
5.4 Sample Data and Queries ---------- 26
Tables:---------- 26
Sample Queries:---------- 26
Chapter 6----------**27**
Application Features and Scenario ---------- 27
6.1 User Interface Design---------- 27
6.2 Login and Authentication System ---------- 27
Scenario 1---------- 28
6.3 Timetable Viewing (Students & Teachers)---------- 30
6.4 Admin Panel Overview---------- 32
Scenario 2---------- 34
Scenario 3---------- 35
Scenario 3---------- 37
Scenario 4---------- 38
6.5 Responsiveness and Cross-Device Compatibility---------- 40
Chapter 7----------**41**
Project Workflow and Implementation---------- 41
7.1 Project Workflow ---------- 41
7.2 GitLab / Version Control Workflow ---------- 43
7.3 Gantt Chart and TimeLine ---------- 45
Chapter 8----------**47**
Testing and Debugging ---------- 47
8.1 Testing Approach ---------- 47
Chapter 9----------**49**
Data Injection into the Database ---------- 49
9.1 Overview---------- 49
9.2 Tools Used---------- 49
9.3 Injected Data Tables---------- 49
9.4 Data Injection Scripts ---------- 50
9.5 Data Verification---------- 50
9.6 Benefits of Custom Injection---------- 51
Chapter 10 ----------**52**

Future Work and Recommendations---------- 52
10.1 Features to Add in Future---------- 52
Conclusion ----------**55**
References----------**56**

List of figures

Figure 0-1: Traditional timetable. 10
Figure 0-1: PDF timetable sent by the delegates. 17
Figure 0-1: Users roles and permissions. 20
Figure 0-1: Architecture Diagram 21
Figure 0-2: Tailwind CSS. 22
Figure 0-3:Technology used (2) 23
Figure 0-4: Technology used (3) 23
Figure 0-5: Technology used (4) 23
Figure 0-1: Entity relationship diagram 24
Figure 0-1: Login Page 27
Figure 0-2: Send Reset Request Page 28
Figure 0-3: Sent Request Alert Page 28
Figure 0-4: Received Email 29
Figure 0-5: Reset Password Page 29
Figure 0-6: Reset Password Successful Alert Response Page 29
Figure 0-7: Student Schedule Page 30
Figure 0-8: Teacher schedule page 31
Figure 0-9: Notification Bell 32
Figure 0-10: Dashboard Page 33
Figure 0-11:Course Session Dashboard 33
Figure 0-12: Manage schedules 34
Figure 0-13: Add New Schedule 35
Figure 0-14: Manage Courses page 35
Figure 0-15: Add New Course Popup Page 36
Figure 0-16: Manage Users Page 37
Figure 0-17: Add New User Popup Page 37
Figure 0-18: Manage Notifications Page 38
Figure 0-19: Exam Notification Creation Popup Page Option 39
Figure 0-20: Received Email Format 39
Figure 0-21: Schedule on Mobile 40
Figure 0-22: Responsive Sidebar 40
Figure 0-1: Workflow 42
Figure 0-2: GitLab Repository and Branching Strategy 43
Figure 0-3: : Snapshot Commit History Backend 44
Figure 0-4: Snapshot Commit History Frontend 44
Figure 0-5: General Git Workflow 45
Figure 0-6: Gantt Chart 46
Figure 0-1: Http Request Test (Get) 47
Figure 0-2: Database Framework Usage 48
Figure 0-3: Console Debugging 48
Figure 0-1: CSV injection 49
Figure 0-2: Python + CSV 49
Figure 0-0-3: A sample of inserted data (.CSV file) 50
Figure 0-1: Remember-me Feature Page 52
Figure 0-2: KeyCloak Logic 53

List of Tables

Table 1: Comparaison Table.....16
Table 2: User Role and Permissions.....20
Table 3: Sample User Data.....26
Table 4: Sample Courses Data.....26
Table 5: Sample Class-Group Data.....26

Chapter 1
Introduction

1.1 Project Background:

University students and instructors often face challenges in accessing up-to-date class schedules, especially when changes occur due to unforeseen circumstances. Traditional methods—such as printed timetables, verbal announcements, or manually updated spreadsheets—are inefficient and prone to delays or miscommunication. This inspired the development of a digital platform that allows students and teachers to consult their schedules in real time and receive immediate notifications about any changes.

As a third-year student in Computer and Communication Network Engineering (CCNE) at the **Faculty of Technology, Lebanese University – Saida Branch**, I wanted to create a meaningful project that directly addresses a problem faced by the academic community, particularly in public Lebanese universities and schools. The idea originated from observing the difficulties students encounter when trying to keep track of schedule updates, especially during sudden changes, holidays, or instructor unavailability.

Schedule - CCNE-S5(En)- 2023-2024

Week 11

		8:00---9:15	9:30---10:45	11:00---12:15	13:00---14:15	14:30---15:45
Monday 20-Jan-00	A1	Web Development-(TD)-Dr. Z. Farhat	VHDL Programming -(C)-Dr. M. Alwan	VHDL Programming -(TD)-Dr. M. Alwan	WLAN and Security-(C)-Dr. W. Fahes	WLAN and Security-(TD)-Dr. W. Fahes
	A2					
	B1					
	B2					
Tuesday 21-Jan	A1	OptoElectronics Systems-(C)-Dr. A. Saleh	Microwaves and Teledetection-(C)-Dr. M. Khatib	Microwaves and Teledetection-(C)-Dr. M. Khatib	Propagation and Antennas-(C)-Dr. A. Saleh	VHDL Programming -(Lab)-Dr. M. Alwan
	A2					
	B1					
	B2					
Wednesday 22-Jan	A1	Network Administration-(TD)-Dr. A. Ismaiil	Propagation and Antennas-(Lab)-Dr.Hanna Farhat	OptoElectronics Systems-(Lab)-Dr.Michel Khoury		French -(TD)- R. H. Chehade
	A2					
	B1		OptoElectronics Systems-(Lab)-Dr.Michel Khoury	Propagation and Antennas-(Lab)-Dr.Hanna Farhat	French -(TD)- R. H. Chehade	
	B2					
Thursday 23-Jan	A1	Propagation and Antennas-(C)-Dr. A. Saleh	Microwaves and Teledetection-(TD)-Dr. M. Khatib	Microwaves and Teledetection-(TD)-Dr. M. Khatib	OptoElectronics Systems-(TD)-Dr. A. Saleh	
	A2					
	B1					
	B2					
Friday 24-Jan	A1			Break		
	A2					
	B1					
	B2					

Figure 0-1: Traditional timetable.

1.2 Problem Statement:

There is currently no centralized digital system within our faculty that allows students and instructors to consult class schedules online in real time or receive immediate alerts about changes. Timetable updates are often communicated informally, leading to confusion, missed classes, and lack of coordination. This absence of an automated system negatively affects time management and the overall educational experience.

1.3 Project Objectives:

The main objectives of this project are to:

- Develop a responsive web application that provides real-time access to university timetables.
- Allow administrators to easily update schedules and notify users instantly of any changes.
- Offer role-based access (admin, student, teacher) with tailored functionalities for each user.
- Create a user-friendly interface accessible from any device.
- Improve communication and organization in educational institutions through digital transformation.

1.4 Project Scope:

This project is focused on creating a full-stack web platform tailored for universities and public educational institutions in Lebanon. The initial implementation is based on the scheduling needs of the **Faculty of Technology – Lebanese University, Saida Branch**. The system includes:

- A frontend interface built using **React and TypeScript**
- A backend developed in **Node.js**
- A **PostgreSQL** database for storing user and schedule data
- A notification system for schedule changes

While the system is designed for use within our faculty, it can be extended and adapted for other branches or public schools across the country.

1.5 Target Users:

The platform is designed to serve the following user groups:

- **Students**: to check their daily or weekly schedules, and receive real-time notifications for class changes or cancellations.
- **Teachers/Instructors**: to view their teaching schedule and be notified of any updates or room changes.
- **Administrators**: to manage course schedules, assign instructors, and send notifications through a dedicated dashboard.

The system is especially aimed at public Lebanese institutions, where digital tools are often lacking or underdeveloped.

1.6 Agile Development Methodology:

The development of this project was guided by the principles of the Agile methodology, a flexible and iterative software development approach that emphasizes incremental progress, collaboration, and continuous feedback. Although the project was conducted individually, Agile practices were simulated through regular planning, development cycles, and weekly meetings with the project supervisor.

Iterative and Incremental Development

The development process followed an iterative model across a three-month period, from April to early July 2025. The work was divided into weekly cycles that functioned as informal sprints, each focused on specific features or components of the system. This approach allowed for manageable progress, easy adaptation to changes, and continuous refinement of the

product.

Weekly Meetings and Feedback

Weekly review meetings were held with the supervisor, Eng. Mohammad Bahlak, typically on weekends. These meetings served as sprint reviews where current progress was presented and feedback was received. At the beginning of the project, two initial meetings were also held with Mrs. Reem Bahlak alongside Mr. Mohammad, during which the project scope and objectives were validated. Subsequent meetings focused solely on technical development and implementation tracking.

Solo Agile Simulation

As a solo developer, I took on the roles typically distributed among team members in Agile teams, including planning, development, testing, and reviewing. At the beginning of each week, I set clear goals and tasks, maintaining a checklist to track progress and priorities. This self-managed workflow ensured focus and accountability throughout the project lifecycle.

Task Planning and Workflow

The first two weeks of development were dedicated to designing and implementing the frontend using React and TypeScript. Once the user interface was completed, the next phase focused on database design using PostgreSQL. This was followed by the development of backend functionalities with Node.js and Express, including route definitions, controller logic, and API endpoints. Each major module (e.g., authentication, scheduling, admin tools) was developed in focused weekly iterations.

Although no formal Agile management tool such as Trello or Jira was used, GitLab served as the central repository for version control. While GitLab Issues were not actively utilized,

the project repository was regularly updated through commits that reflected the step-by-step development of features and fixes.

Benefits of Agile Use

Simulating Agile practices in a solo context proved highly beneficial. Regular supervisor feedback helped correct the course early and often, minimizing the risk of major errors accumulating. The iterative approach also allowed continuous integration and testing of components, making it easier to identify and resolve bugs early in the development process.

Chapter 2
Literature Review / Existing Solutions

2.1 Overview of Existing Systems:

Many universities around the world use learning management systems (LMS) or schedule management platforms such as **Moodle**, **Google Calendar**, **Blackboard**, or custom-built institutional portals. These platforms often integrate course materials, assignments, and class schedules. However, in many cases—especially in **public universities in Lebanon**—such solutions are either unavailable, underutilized, or lack essential features like real-time updates and instant notifications.

In some institutions, timetables are still distributed as **PDF documents or Excel sheets**, posted on bulletin boards or shared via messaging apps like WhatsApp. These static formats are not easily updated or maintained, and users must constantly check manually for any changes. This process is inefficient and prone to human error, especially when instructors change availability or classroom resources are rescheduled.

Table 1: Comparaison Table

Feature	Your Web App	University PDF via WhatsApp
Real-Time Updates	Yes – instantly updated online	**No – requires manual re-sending**
Notifications	Yes – instant alerts	**No – depends on someone sending it**
Accessibility	Any device with internet	**Only if you receive the message**
Customization	Tailored to university structure	**Fixed, general format**
User Roles	Admin, Student, Teacher views	**One static version for all users**
Centralized System	All data in one platform	**Scattered across messages/files**
Update Frequency	Real-time, by admins	**Occasional, manually shared**
Reliability	High – structured and secure	**Low – prone to miscommunication**
Offline Availability	Requires internet	**PDF can be downloaded**
Popularity	Growing in target faculty	**Widely used (but outdated)**



Figure 0-1: PDF timetable sent by the delegates.

2.2 Limitations in Existing Solutions:

Existing systems, whether digital or manual, present several shortcomings:

- **Lack of Real-Time Updates**: Changes to schedules are not reflected in real time, which leads to students showing up to canceled or rescheduled classes.
- **Poor Communication**: Instructors and students often rely on indirect communication (emails, group chats) for updates, which is not centralized or reliable.
- **Inaccessible Platforms**: Some systems require university email access or credentials not all students have or regularly check.
- **Limited Customization**: Many platforms are too general and not tailored to the unique organizational structures of Lebanese public universities.
- **No Notification System**: Few platforms offer direct push or in-app notifications that alert users to schedule changes.

2.3 Motivation for This Project

The gap between available solutions and the needs of students in **Lebanese public institutions**—especially at the **Faculty of Technology, Lebanese University in Saida**—is clear. Many students are unaware of schedule changes until it's too late, and teachers often struggle to inform multiple classes quickly.

This project aims to bridge that gap by offering a **custom-built, web-based scheduling platform** that:

- Displays real-time timetable data for each user type (student, teacher, admin)
- Allows quick administrative updates
- Sends instant notifications upon changes
- Can be adapted to other faculties or public schools with minimal configuration

Unlike general-purpose tools like Google Calendar or Telegram groups, this system is built with the **specific needs of local academic structures** in mind, and it integrates **role-based access, real-time updates, and institutional scheduling logic**.

Chapter 3
System Analysis and Requirements

3.1 Functional Requirements:

The system is designed to manage university schedules with different user roles and real-time updates. Below are the core functionalities:

For Students:

- View personal class schedule by week or day
- Receive notifications when their schedule changes
- View course names, instructors, and classroom details

For Teachers:

- View assigned courses and schedule
- Get notified of changes to their teaching schedule
- Filter schedule by day

For Administrators:

- Add/edit/delete courses, rooms, and time slots
- Assign instructors to courses
- Manage user accounts (students, teachers)
- Send real-time notifications to users
- View complete schedule for all users

3.2 Non-Functional Requirements

These requirements define the system's quality attributes and performance expectations:

- **Responsiveness**: The application should work smoothly across desktops, tablets, and mobile devices
- **Real-Time Notification**: Schedule changes should instantly reflect in the UI and notify users
- **Security**: Role-based access control and secure authentication (e.g., JWT)
- **Scalability**: The system should support hundreds of users and potential expansion to other faculties or institutions
- **Usability**: The user interface should be intuitive and easy to navigate even for non-technical users
- **Availability**: The application should remain accessible with minimal downtime

3.3 Use Case Diagrams:

Below is a textual description of the use cases that can be illustrated with diagrams in the appendix:

Student Use Cases:

- Login
- View timetable
- Receive notifications

Teacher Use Cases:

- Login
- View teaching schedule
- Get notified of updates

Admin Use Cases:

- Login
- Manage users (add/edit/delete)
- Manage courses and time slots
- Assign instructors to courses
- Send notifications

3.4 User Roles and Permissions:

Table 2: User Role and Permissions

Role	Permissions
Student	View schedule, receive notifications
Teacher	View assigned schedule, receive notifications
Admin	Full access: create/edit/delete courses, users, rooms, and time slots; manage notifications

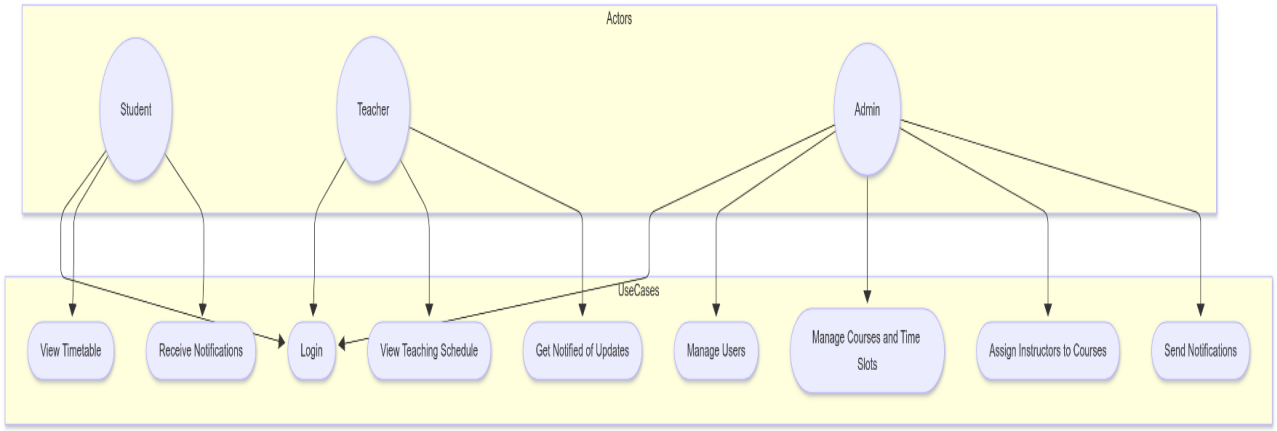


Figure 0-1: Users roles and permissions.

Chapter 4
System Design and Architecture

4.1 System Overview:

This project presents a comprehensive web-based timetable management system that allows students and teachers of the Lebanese University – Faculty of Technology, Saida Branch, to consult their schedules in real time. The system also notifies users of any schedule changes such as cancellations, room changes, or rescheduled sessions. The platform was developed using modern web technologies and follows a modular, scalable architecture suitable for academic institutions.

The system consists of three major components:

- **Frontend** built with React and TypeScript, offering dynamic and user-friendly interfaces.
- **Backend API** built with Node.js (Express), handling authentication, scheduling logic, notifications, and database interactions.
- **Database** using PostgreSQL to manage users, schedules, courses, and notification data.

4.2 Architecture Diagram:

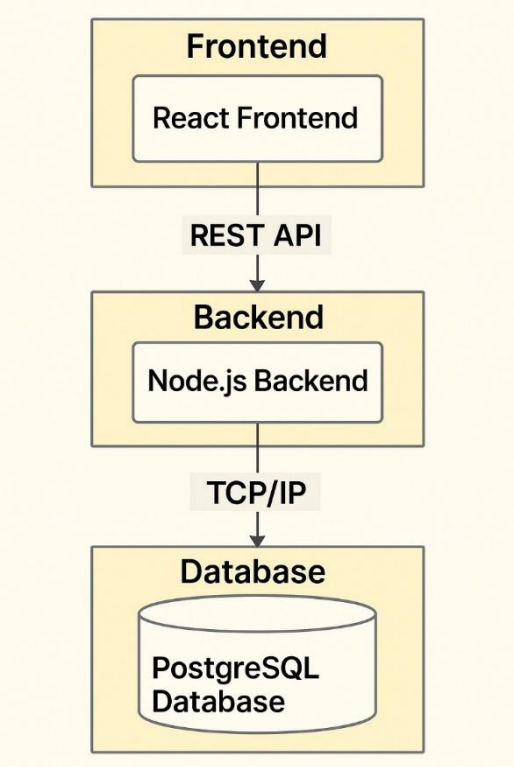


Figure 0-1: Architecture Diagram

4.3 Component Diagram:

- **Frontend Pages**: LoginPage, ResetPasswordPage, UserSchedulePage, TeacherSchedulePage, AdminDashboardPage, AdminManageCoursesPage, etc.
- **Services (API communication)**: authApi.ts, userApi.ts, scheduleApi.ts, notificationsApi.ts, etc.
- **Backend Controllers**: authController.js, userController.js, scheduleController.js, notificationController.js, etc.
- **Middleware**: Authentication and admin access validation
- **Database Models**: Users, Courses, ClassGroups, Schedules, Notifications, etc.

4.4 Technologies and Tools Used:

React & TypeScript

Used to build the frontend user interface. React's component-based architecture improves maintainability, while TypeScript ensures type safety, reducing bugs and enhancing development efficiency.

Tailwind CSS: A utility-first CSS framework used to style the application with responsive and clean designs. Tailwind allowed for fast UI development by applying pre-defined classes directly in the markup, reducing the need for writing custom CSS from scratch.



Figure 0-2: Tailwind CSS.

Node.js (Express)

Serves as the backend technology to create RESTful APIs. Express simplifies request routing, middleware integration, and scalable backend logic, making it ideal for handling user authentication, password resets, and schedule updates.

PostgreSQL

A powerful, open-source relational database used for storing all structured data including users, schedules, class groups, and notifications. Its support for ENUM types allowed strong typing for roles, sections, schedule types, etc.



Figure 0-3:Technology used (2)

- **Git**: Version control to manage source code across development stages.



Figure 0-4: Technology used (3)

- **Postman**: Used for testing backend APIs during development.
- **Thunder Client (VS Code extension)**: Used for lightweight and fast API testing during backend development.

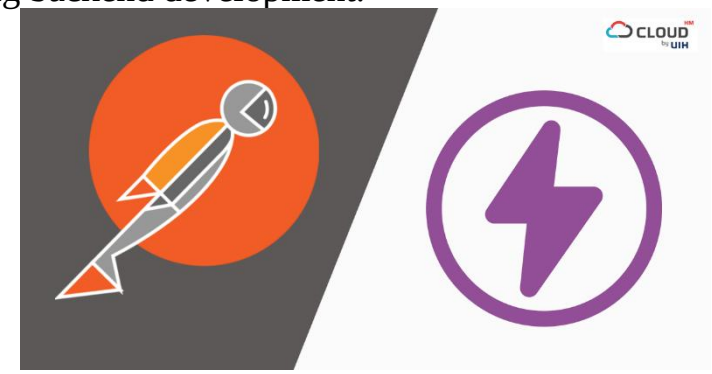


Figure 0-5: Technology used (4)

Chapter 5
Database Design

5.1 Entity-Relationship Diagram:

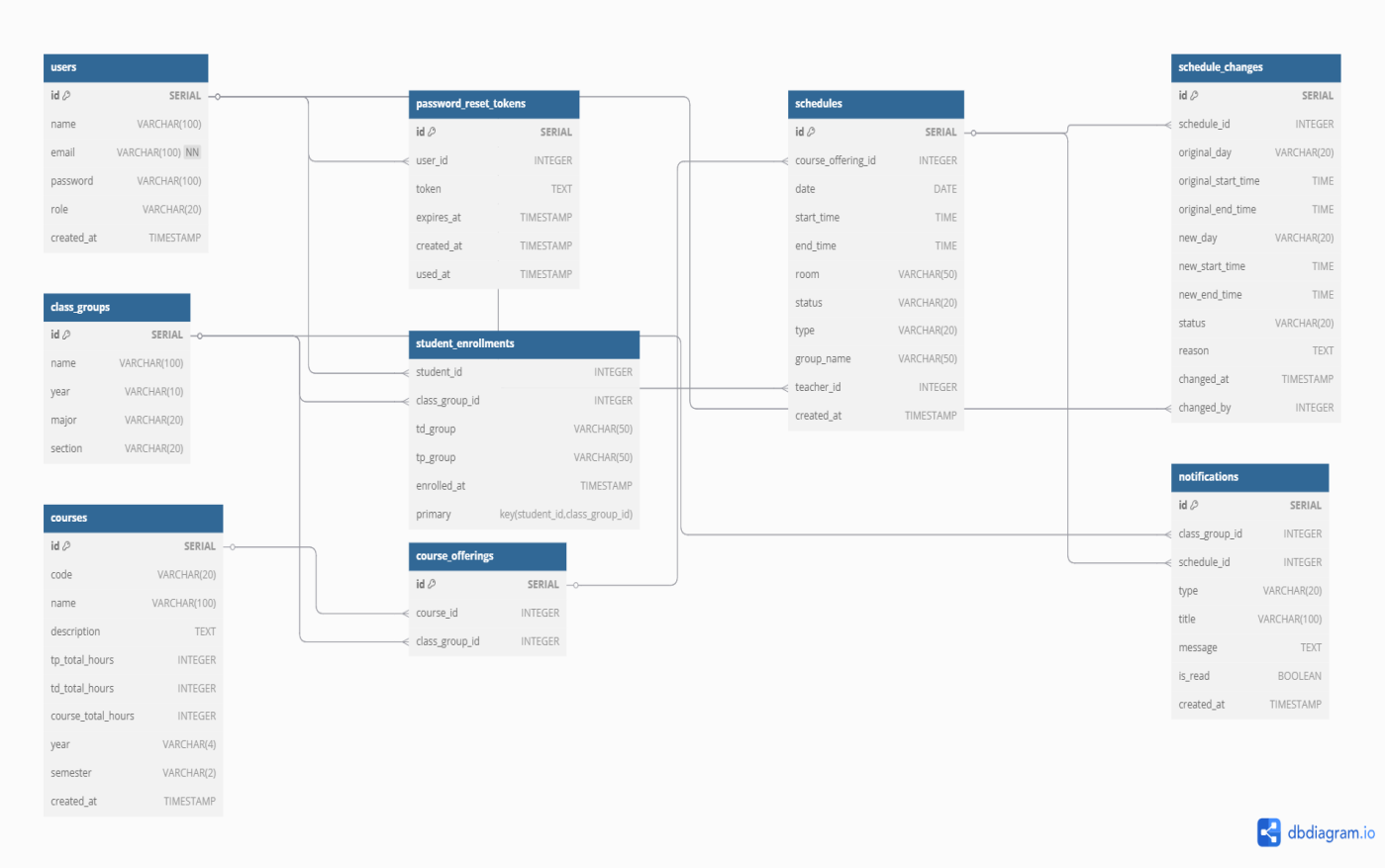


Figure 0-1: Entity relationship diagram

Entities:

- Users
- ClassGroups
- StudentEnrollments
- Courses
- CourseOfferings
- Schedules
- ScheduleChanges
- Notifications
- PasswordResetTokens

5.2 Tables and Relationships:

- **users**: Stores login credentials and user roles (student, teacher, admin)
- **class_groups**: Defines student batches by year, major, and section
- **student_enrollments**: Many-to-many relationship between users and class groups
- **courses**: Contains general course data
- **course_offerings**: Assigns teachers to courses and groups
- **schedules**: Timetable entries for each course offering
- **schedule_changes**: Tracks modifications to original schedules
- **notifications**: Stores user-visible alerts for schedule updates
- **password_reset_tokens**: Handles secure password reset functionality

5.3 Database Schema Explanation:

The database design focuses on normalization, clarity, and extensibility:

- **ENUMs** are used for predefined values (roles, days, session types, etc.), ensuring data consistency.
- **Foreign keys** ensure referential integrity across related entities.
- **Composite keys** in student_enrollments and course_offerings prevent duplicate associations.
- **Schedule changes** are decoupled from the main schedule to allow logging and rollback.

5.4 Sample Data and Queries:

Tables:

Table 3: Sample User Data

Users id	name	Email	role
1	Ahmad Ramadan	ahmad.ramadan2005f@gmail.com	student
2	Prof. Hannah Farhat	HannahFarhat@univ.edu	teacher
3	Admin	admin@univ.edu	admin

Table 4: Sample Courses Data

Courses id	code	name	description	Tp total hours	Td total hours	Cours total hours	Course Total Hours	Year	Semester
1	XX	Network	description	10	15	10	35	2025	1

Table 5: Sample Class-Group Data

Class_groups id	Name	Year	Major	Section
1	2 MIS english	2	MIS	english

Sample Queries:

1. **Get upcoming schedule for a user**
   SELECT s.*
   FROM schedules s
   JOIN course_offerings co ON s.course_offering_id = co.id
   JOIN student_enrollments se ON se.class_group_id = co.class_group_id
   WHERE se.student_id = 1 AND s.day = 'Monday';
2. **Insert a notification**
   INSERT INTO notifications (class_group_id, schedule_id, type, title, message)
   VALUES (1, 5, 'schedule_change', 'Change in Network Course', 'Your TD session has been moved to Room 101.');
3. **Get unread notifications for a class group**
   SELECT * FROM notifications
   WHERE class_group_id = 1 AND is_read = false;

Chapter 6
Application Features and Scenario

6.1 User Interface Design:

The user interface was built using **React** with **TypeScript**, ensuring type safety and smooth developer experience. The UI focuses on clarity and accessibility, with three main user roles: **Admin**, **Teacher**, and **Student**.

- **Login Page** – clean design with forgot password link
- **Admin Dashboard** – sidebar navigation for managing users, schedules, courses, and notifications
- **User Schedule Page** – weekly view of class schedules
- **Teacher Schedule Page** – personalized class schedule view for teachers
- **Reset Password Page** – token-based secure reset flow

6.2 Login and Authentication System:

The authentication system is built with **JWT** (JSON Web Tokens). Users log in with their email and password, and a secure token is stored in local storage to manage sessions.

- Role-based access (student, teacher, admin)
- Passwords hashed with bcrypt
- Forgot password functionality using email + token

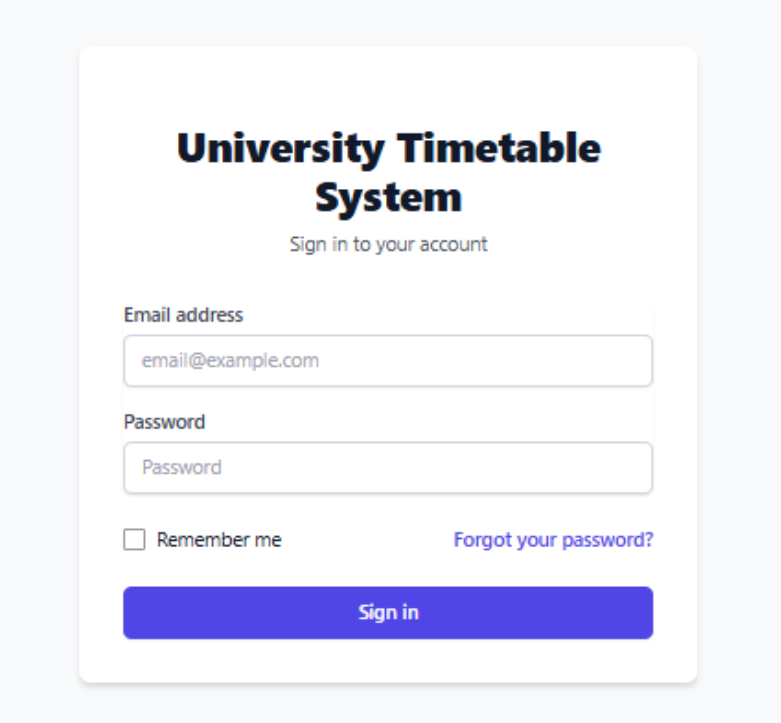


Figure 0-1: Login Page

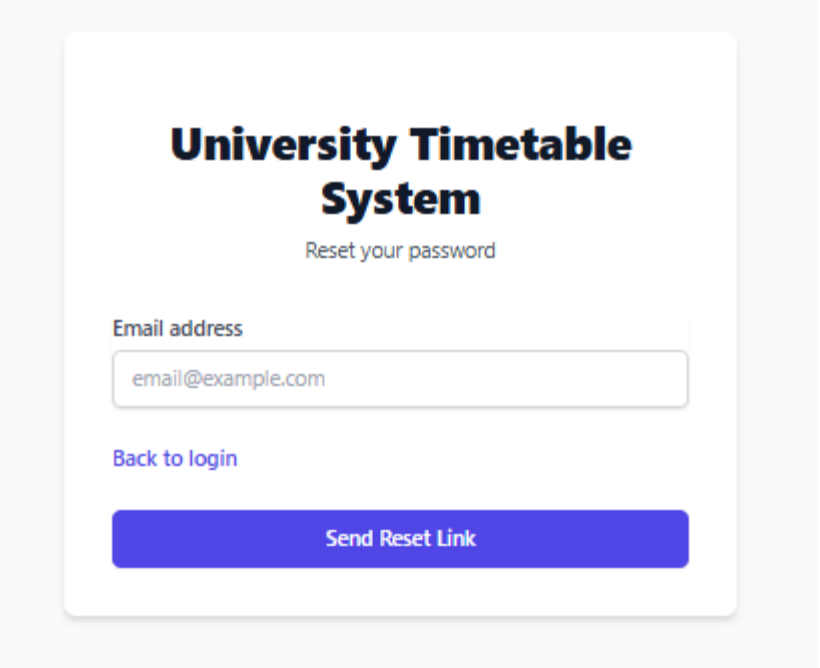


Figure 0-2: Send Reset Request Page

✓ Scenario 1:
logging in, receiving email and changing password:

Rayan forgot her password and is attempting to reset it in order to log in. She navigates to the login page and clicks on the "Forgot Password" option. This takes her to the password reset request page, where she enters her registered email address. Shortly after, she receives a password reset email, opens the provided link, and sets a new password. Once the update is successful, she uses her new credentials to log in to her account.

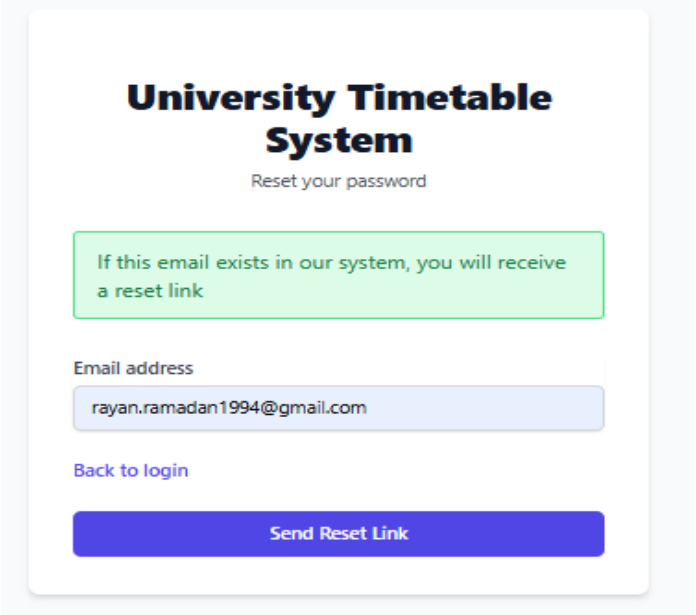


Figure 0-3: Sent Request Alert Page

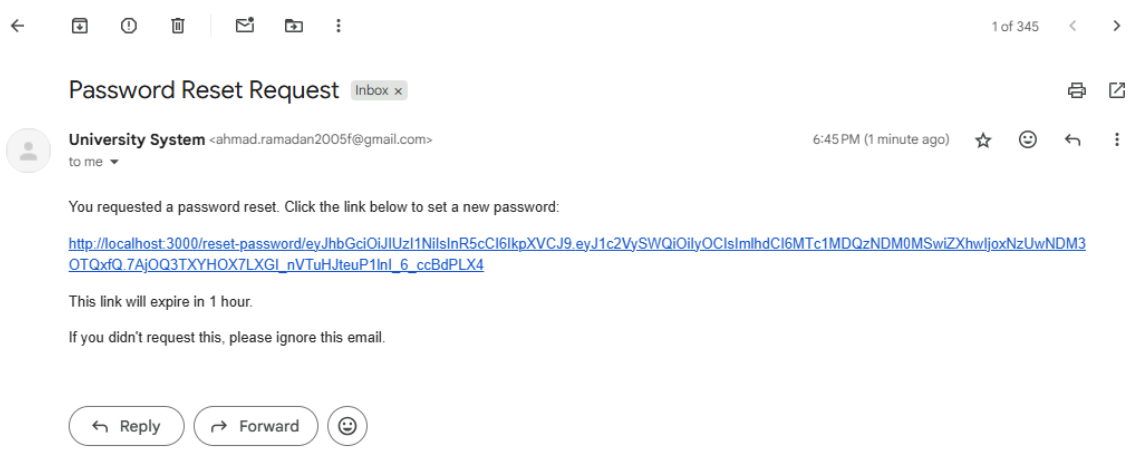


Figure 0-4: Received Email

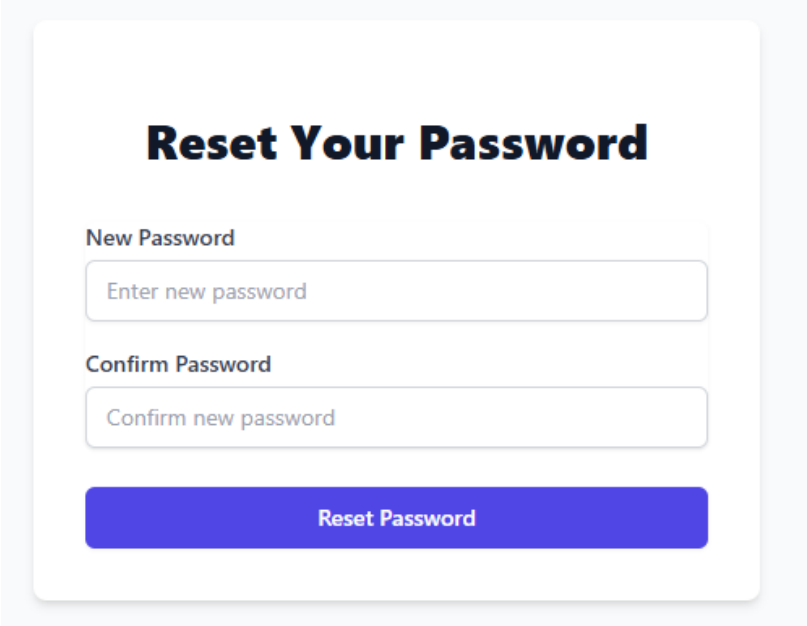


Figure 0-5: Reset Password Page

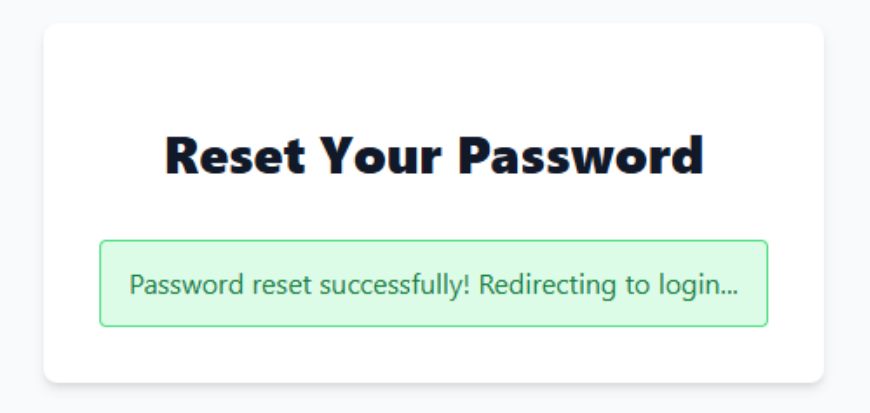


Figure 0-6: Reset Password Successful Alert Response Page

6.3 Timetable Viewing (Students & Teachers):

Users can view their schedules dynamically:

- Students see their enrolled class group schedules.
- The schedule page allows students to filter their timetable by: Academic year, Language, TD group, TP group, Week range.

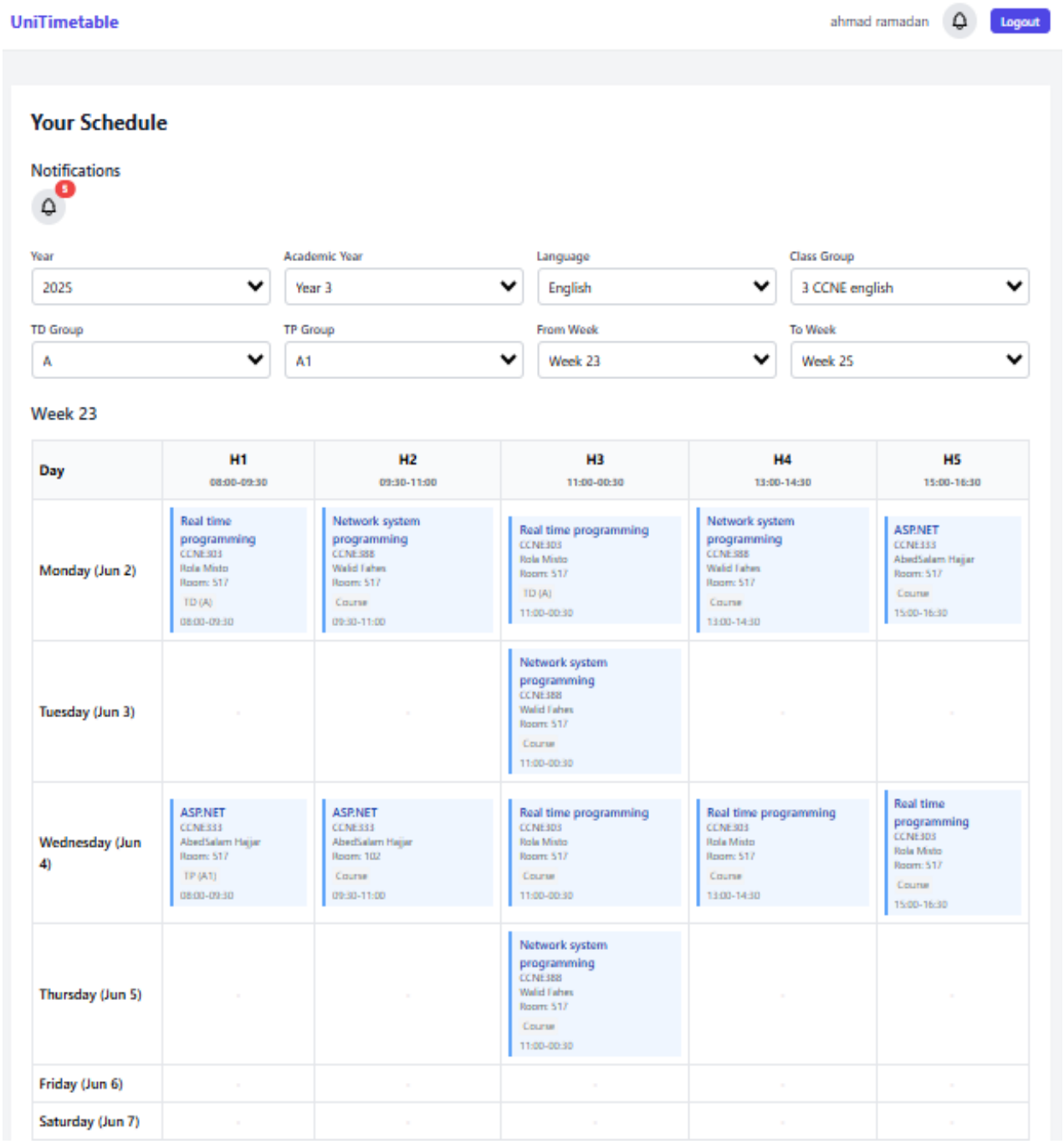


Figure 0-7: Student Schedule Page

- Teachers see classes they are assigned to.
- The schedule page allows Teachers to filter their timetable by: Academic year, Language, TD group, TP group, Week range.

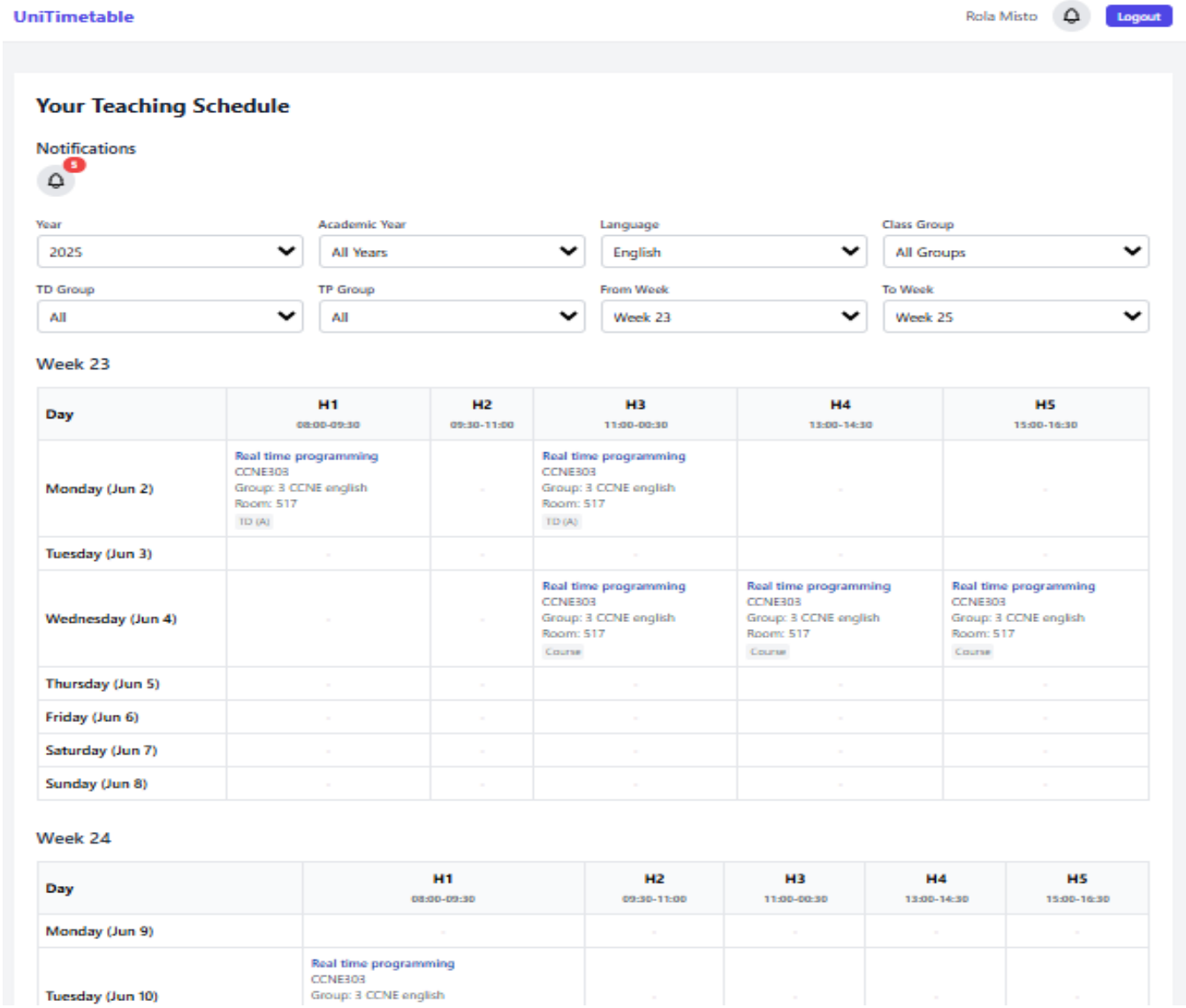


Figure 0-8: Teacher schedule page

- Views Notification Bell when data changes (like cancellations or reschedules)

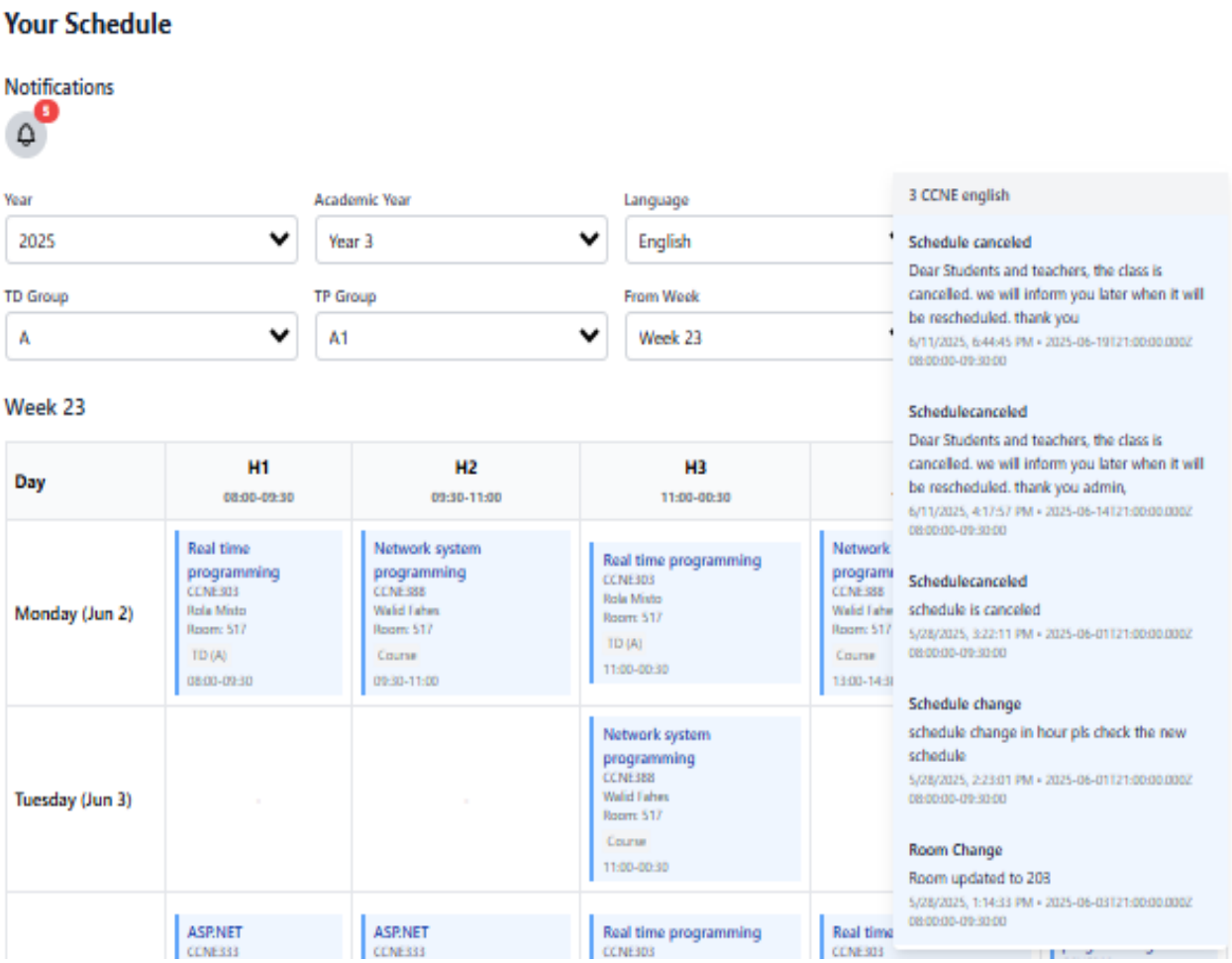


Figure 0-9: Notification Bell

6.4 Admin Panel Overview:

The Admin Panel provides full control over system management through a sidebar-based interface. The panel includes the following sections:

1. Dashboard

- Displays general system metrics and summaries.
- Number of users
- Number of courses in the system
- Some percentages and statistics about the typr of schedule changes.

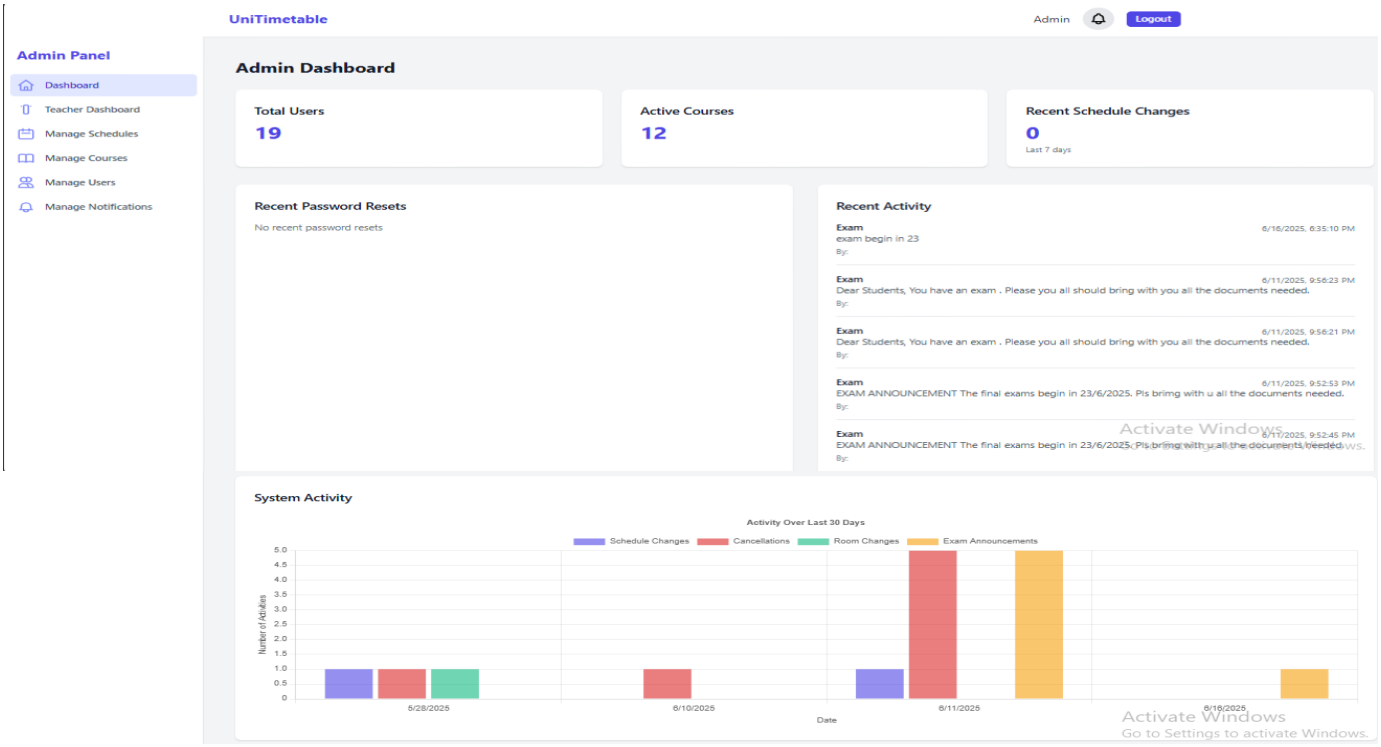


Figure 0-10: Dashboard Page

2. Course Session Dashboard

- Overview of assigned courses and the number of completed sessions versus total sessions in each TP, TD, and course.
- Plus filtering features by course name, semester year and session type.

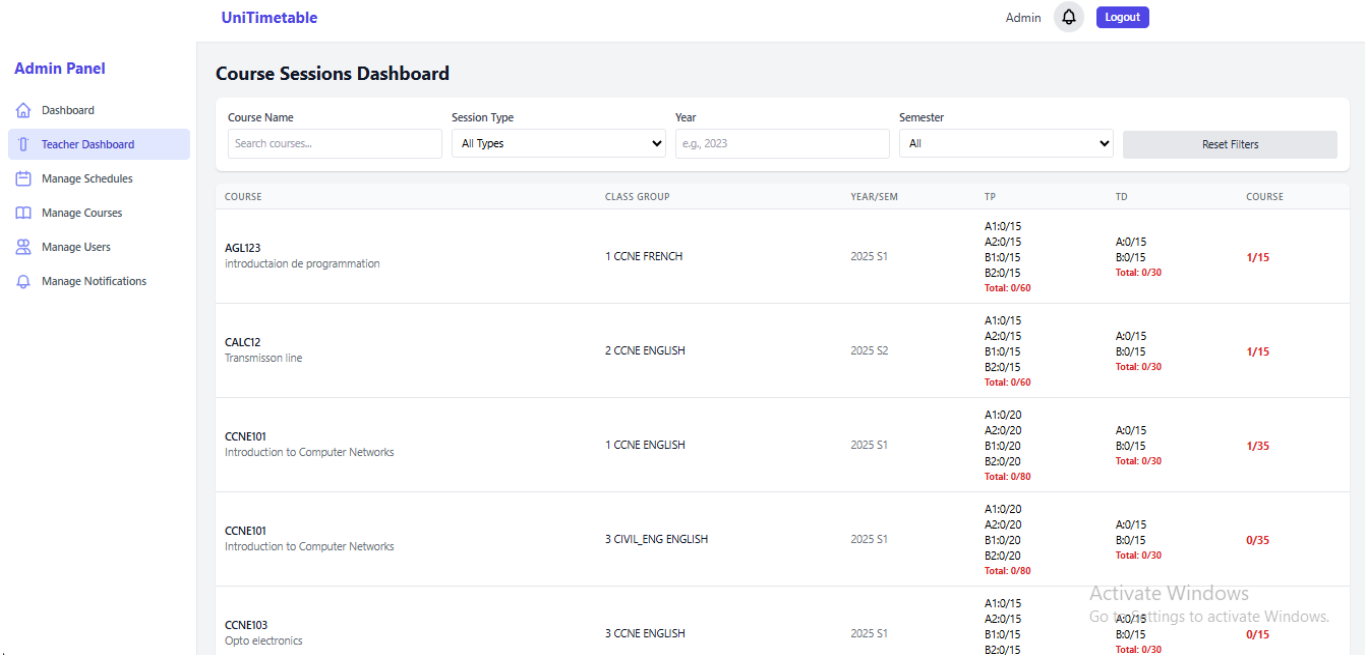


Figure 0-11:Course Session Dashboard

3. Manage Schedules

- Add new sessions : Course, TD (Travaux Dirigés), or TP (Travaux Pratiques) or a course session. Plus, Filtering features by year, major, section, teacher and date range.
- Cancel or reschedule classes with automatic change tracking, Modify or delete existing's.
- View complete weekly schedules per class group.
- Classes should be created with a session duration of **1 hour and 30 minutes**; otherwise, they will not be displayed in the schedule.

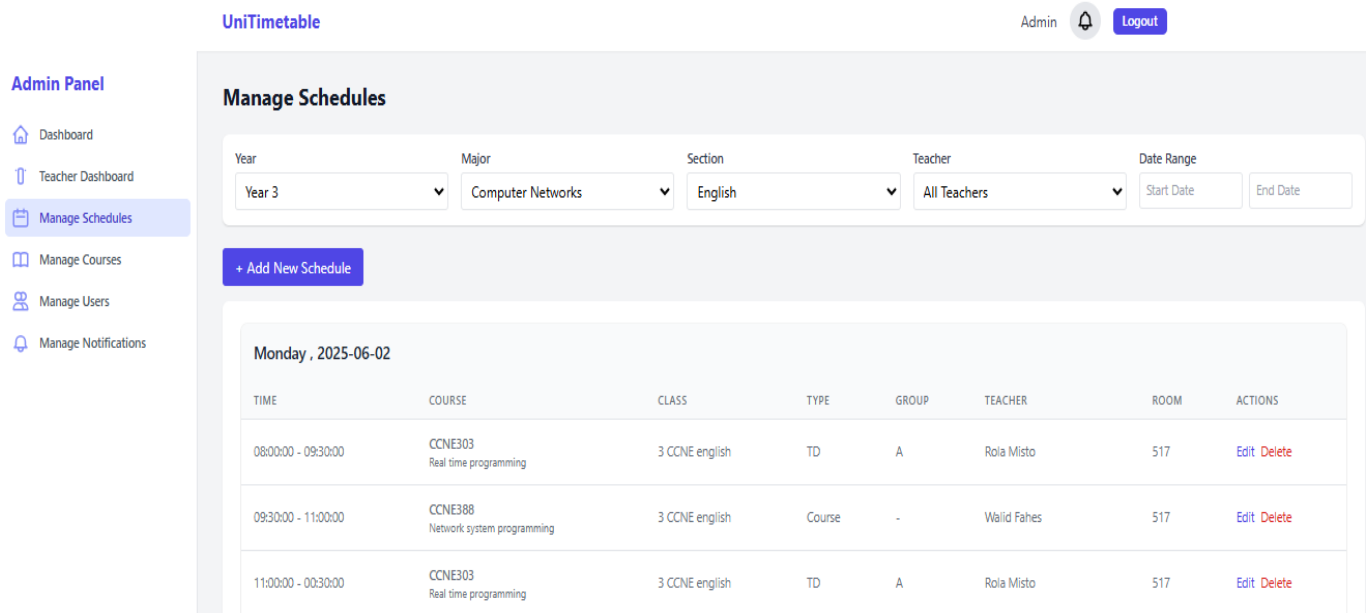


Figure 0-12: Manage schedules

✓ Scenario 2:

Adding a New Class Schedule:

An administrator wants to create a new schedule entry for the ASP.NET (CCNE333) course. In the scheduling form, they select the following details:

- **Course**: ASP.NET (CCNE333) – 3 CCNE English
- **Teacher**: Prof. Abed Salam Hajjar
- **Date**: 2025-07-05
- **Session Type**: TD
- **TD Group**: A
- **Start Time**: 08:00 AM
- **End Time**: 09:30 AM
- **Room**: 517

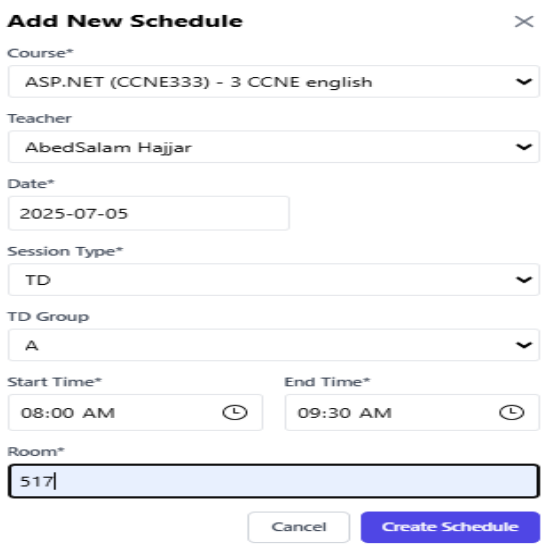


Figure 0-13: Add New Schedule

4. Manage Courses

- Add, edit, or delete courses.
- Assign courses to departments or programs.
- Link courses to available class groups.
- Filtering by course name, year, major, section.

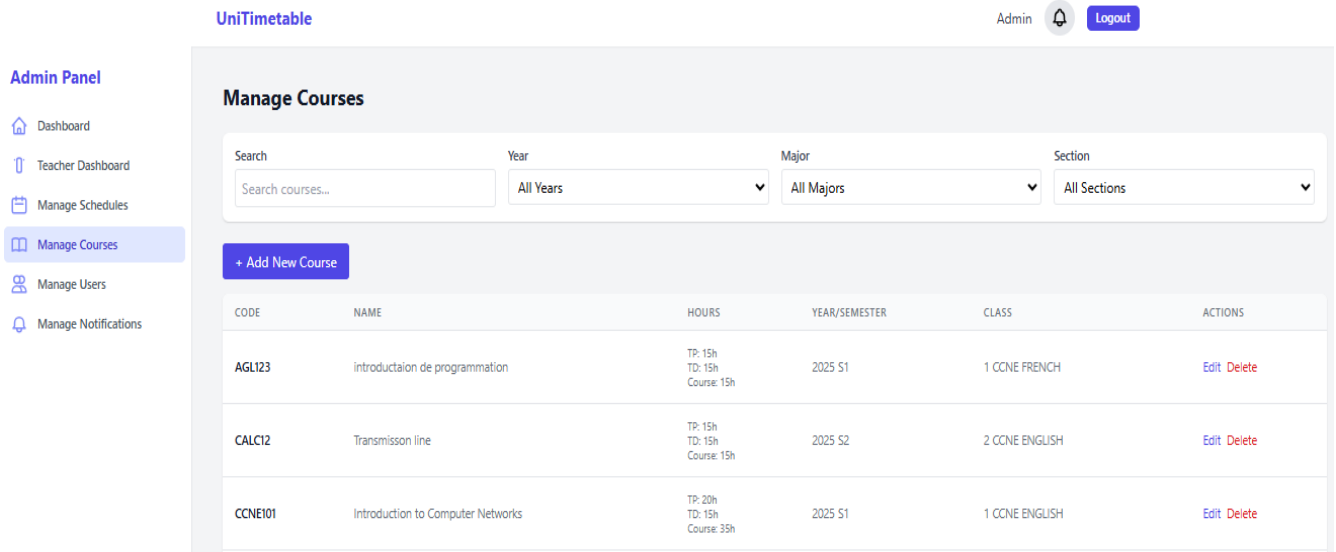


Figure 0-14: Manage Courses page

✓ Scenario 3:

Adding a New Course:

An administrator wants to add a new course to the system for the upcoming semester. They open the **"Add New Course"** form and begin filling in the details:

- **Course Code**: CCNE103
- **Course Name**: Operation System
- **Description**: Operating System
- **TP Hours**: 15
- **TD Hours**: 10
- **Course Hours**: 20
- **Academic Year**: 2025
- **Semester**: S2

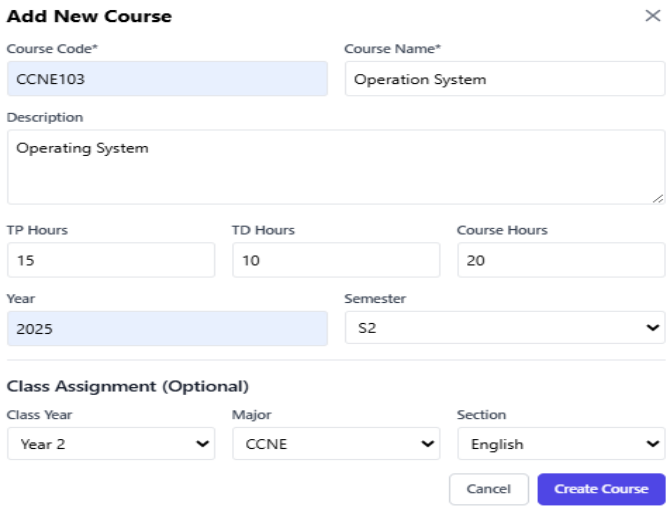


Figure 0-15: Add New Course Popup Page

5. Manage Users

- Add new users (students, teachers, admins).
- Edit or delete user accounts.
- Assign students to class groups.
- Set user roles and access levels.
- Filtering by user name, role, year, major and section.

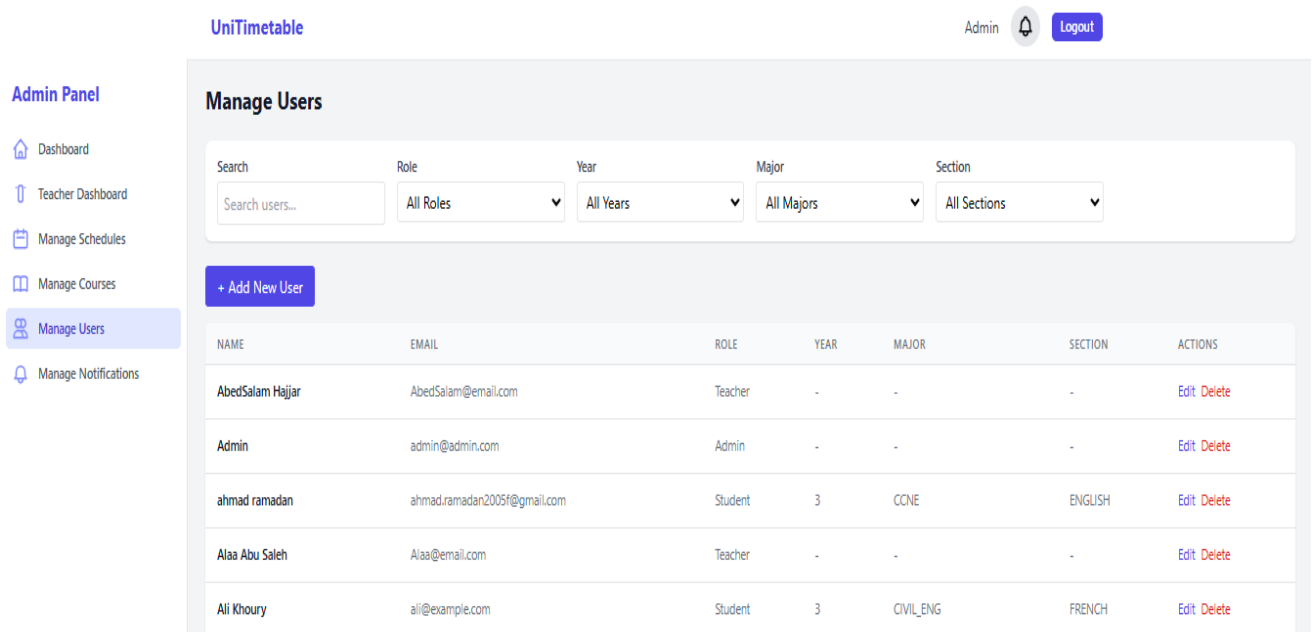


Figure 0-16: Manage Users Page

✓ Scenario 3:

Adding a New User:

An admin fills out the "Add New User" form to register a student named George. They enter his email (Maroun), assign him the role of Student, and select his academic details: Year 1, CCNE major, English section, TD group A, and TP group A1. After setting a password and confirming it, the admin clicks "Create User" to complete the registration.

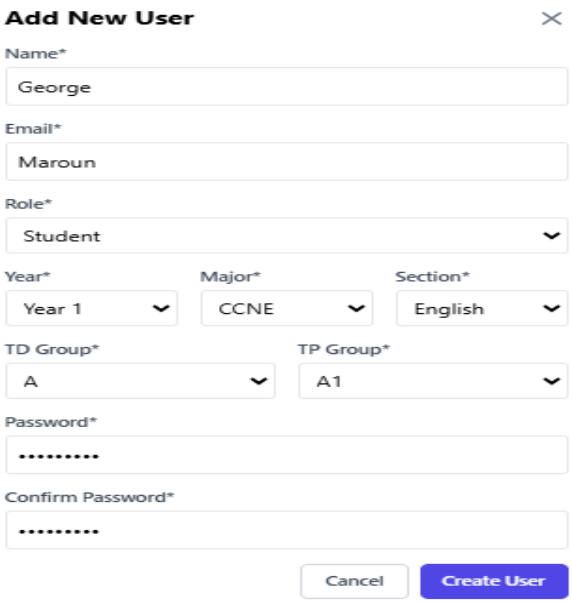


Figure 0-17: Add New User Popup Page

6. Manage Notifications

- Send real-time notifications to users or groups + Send it by email.
- Types of notifications:
  - Room changes
  - Class cancellations
  - Exam announcements
  - Rescheduled sessions
- All notifications are stored in the database and displayed based on user/class group + sent by email for each class/teacher.

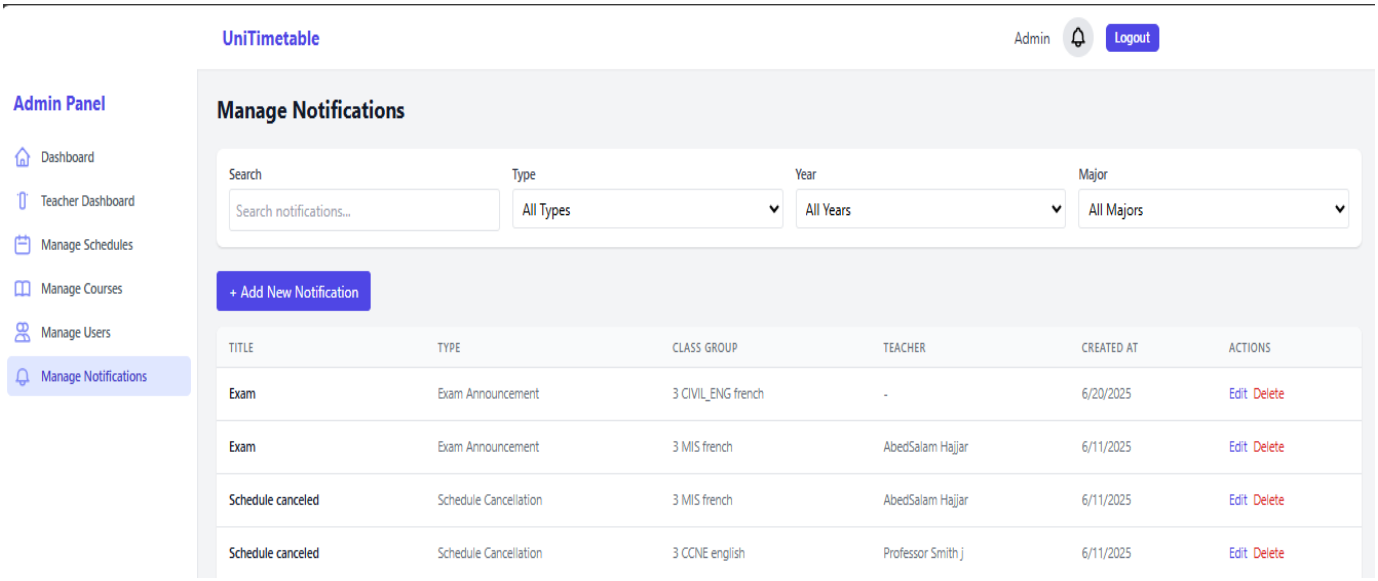


Figure 0-18: Manage Notifications Page

✓ Scenario 4:
Adding a New Notification that will be sent by email:

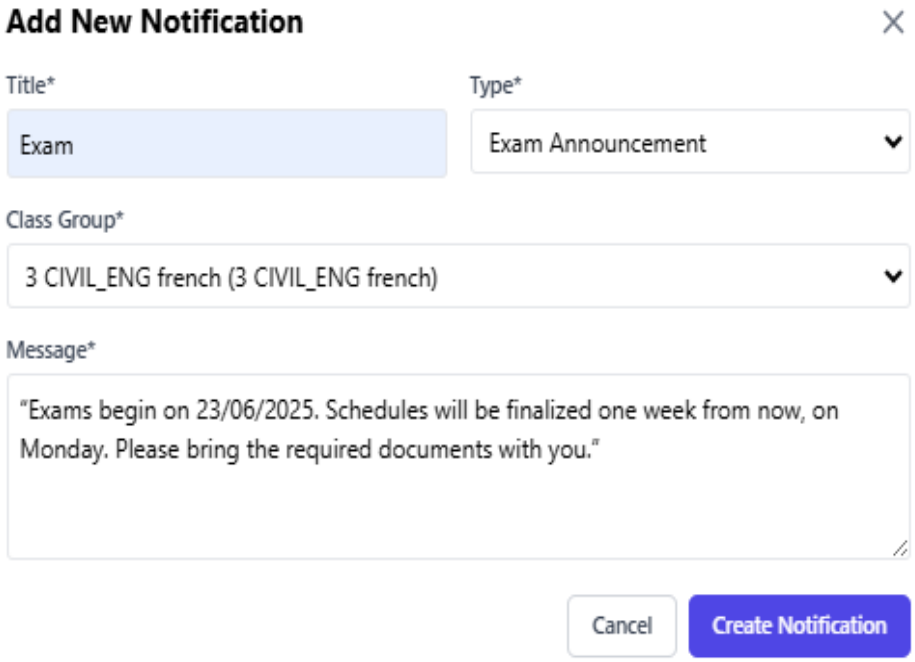


Figure 0-19: Exam Notification Creation Popup Page Option

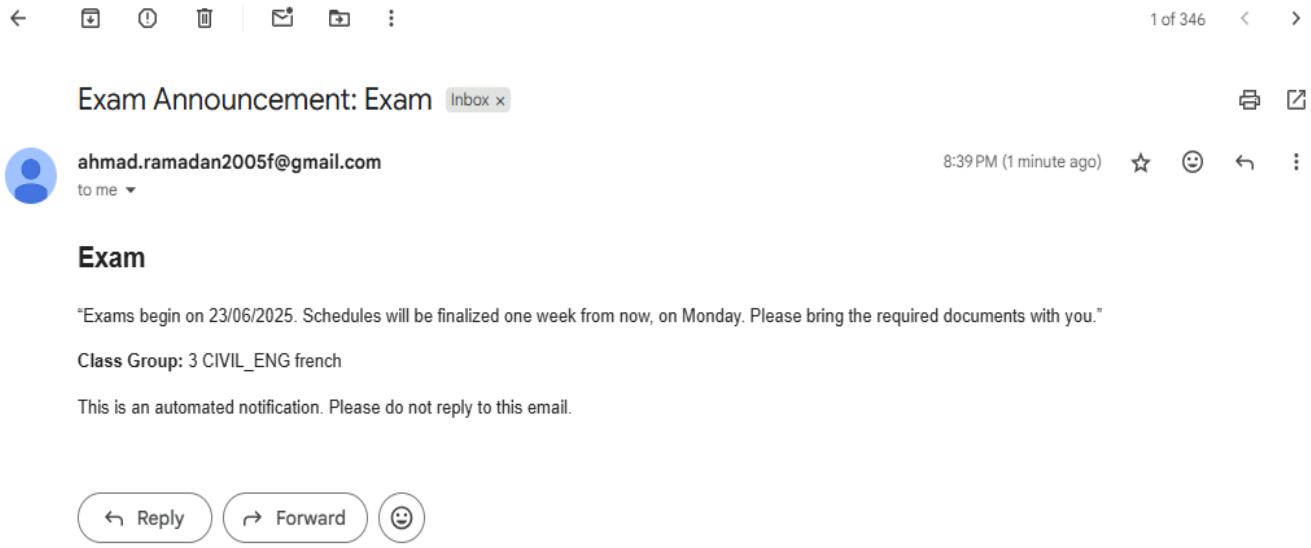


Figure 0-20: Received Email Format

6.5 Responsiveness and Cross-Device Compatibility:

The web application was designed with a mobile-first approach, ensuring that users can access and interact with the platform from any device — whether on a desktop, tablet, or smartphone. Responsive layout techniques using media queries and flex/grid systems in CSS were implemented to adapt UI components dynamically.

- Student schedule on mobile

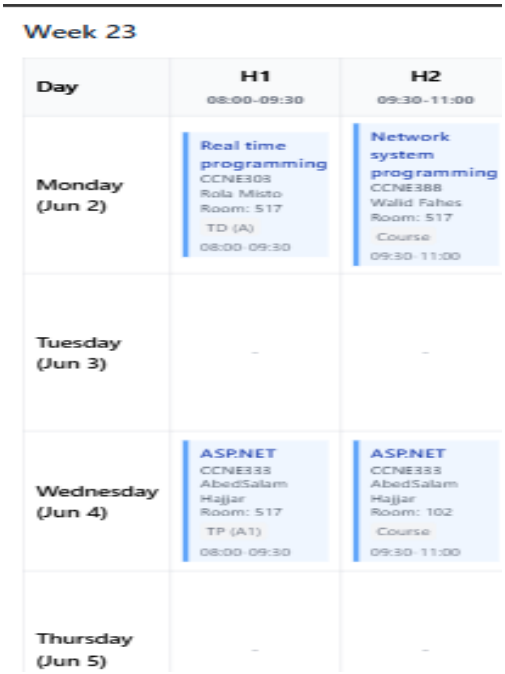


Figure 0-21: Schedule on Mobile

- Admin dashboard on Mobile.

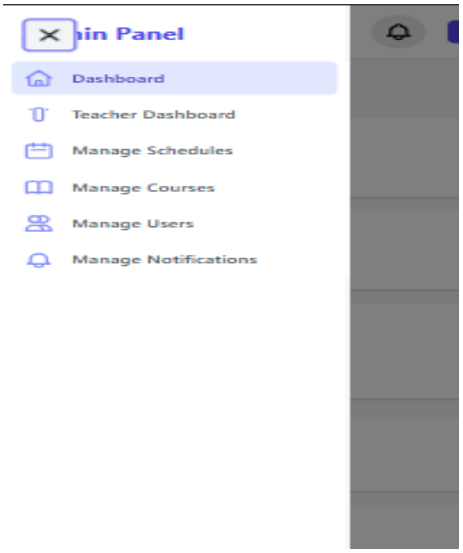


Figure 0-22: Responsive Sidebar

Chapter 7
Project Workflow and Implementation

7.1 Project Workflow:

- **Student Workflow**

When a student logs into the system using their credentials, the backend verifies their role using the users table. Upon successful login, the student is redirected to their personal schedule page. This page allows them to filter their schedule by academic year, language, TD group, TP group, and a week range. The system identifies the student's class_group_id from the student_enrollments table and fetches all matching course_offerings and associated schedules based on the selected filters.
Each class session is displayed on a weekly timetable, showing the course name, room, type (TD, TP, or Course), and instructor. In parallel, the system checks for unread notifications related to the student's class group from the notifications table (where class_group_id = student's class). These notifications are highlighted with a bell icon to ensure the student is informed about any changes or cancellations in real time. This approach eliminates reliance on WhatsApp or manual updates and ensures students always have access to an up-to-date and personalized schedule.

- **Teacher Workflow**

Teachers log in using their university-provided credentials. Upon login, their role is detected from the users table, and they are redirected to the **Teaching Schedule** page. The system retrieves all schedules where the teacher_id matches the logged-in user. The result is filtered by academic week and displayed in a grid format similar to the student view, showing all sessions the teacher is responsible for.
In addition, teachers may receive notifications for schedule changes that concern their assigned sessions. These are fetched from the notifications table by checking if any notification is linked to a schedule_id assigned to that teacher. This ensures instructors stay aware of any last-minute updates or room changes without needing to rely on word of mouth or external communication channels.

- **Admin Workflow**

Administrators access the system via the **Admin Dashboard**, where they manage all core operations related to users, schedules, courses, and

notifications. After authentication, they are redirected to the main dashboard where they can see system statistics such as total users, weekly scheduled sessions, and pending notifications.

- **Course Session Dashboard** offers an overview of assigned courses and tracks the number of completed sessions versus total sessions across TP, TD, and Course types. It also includes filtering options by course name, semester year, and session type.
- In the **Manage Users** section, admins can create student, teacher, or admin accounts. Students are assigned to a class group, with specific TD and TP groups, by inserting records into the student_enrollments table.
- In **Manage Courses**, admins define new courses with details such as course code, name, description, hours, and semester.
- The **Manage Schedule** page allows admins to assign specific courses to class groups using the course_offerings table and schedule the sessions by filling out the schedules table. Sessions can be labeled by type (TD, TP, or Course), date, time, room, and instructor.
- Finally, in **Manage Notifications**, the admin can push alerts regarding schedule changes, room updates, or cancellations. These notifications are inserted into the notifications table and are immediately visible to affected students and teachers via their dashboards.

  Through this centralized role-based workflow, the system ensures real-time coordination, better time management, and an overall improvement in the communication infrastructure within public academic institutions.

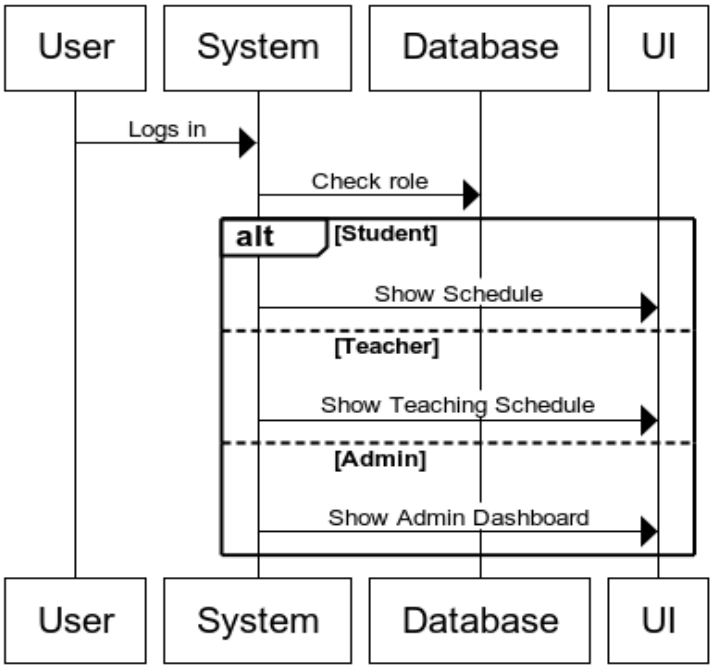


Figure 0-1: Workflow

7.2 GitLab / Version Control Workflow:

For version control, the project used **Git**, with **GitLab** as the central repository and collaboration platform. A **branch-based workflow** was adopted to separate different parts of the application and manage updates efficiently. The main branches included:

- main – the production-ready branch containing stable code.
- frontend-update – used for implementing and updating frontend features.
- backend-update – used for backend development and modifications.

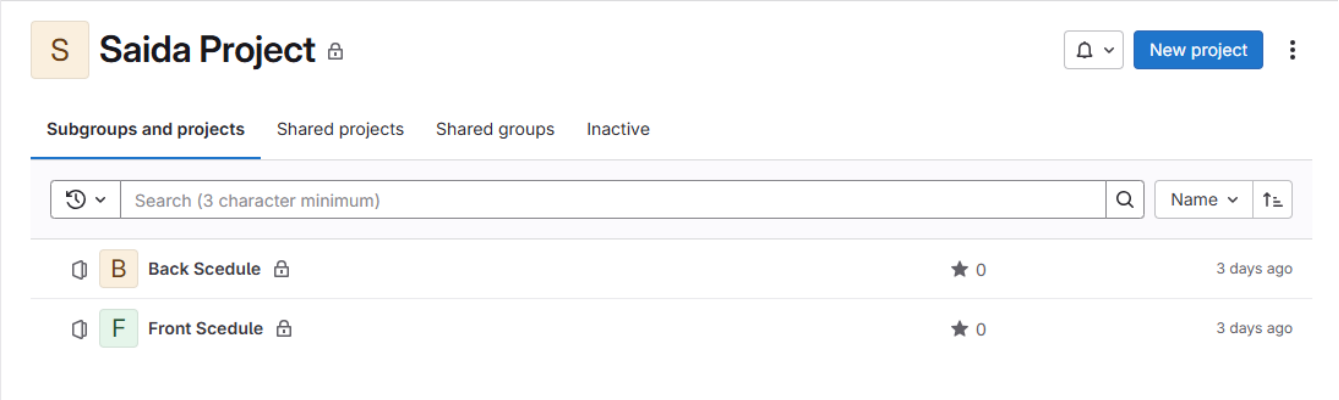


Figure 0-2: GitLab Repository and Branching Strategy

During development, changes were pushed to GitLab regularly using the following workflow:

```
git checkout <branch-name>
git add .
git commit -m "meaningful commit message"
git push origin <branch-name>
```

For example, when making changes to the backend, the backend-update branch was used:

```
git checkout backend-update
git add .
git commit -m "second deployment"
git push origin backend-update
```

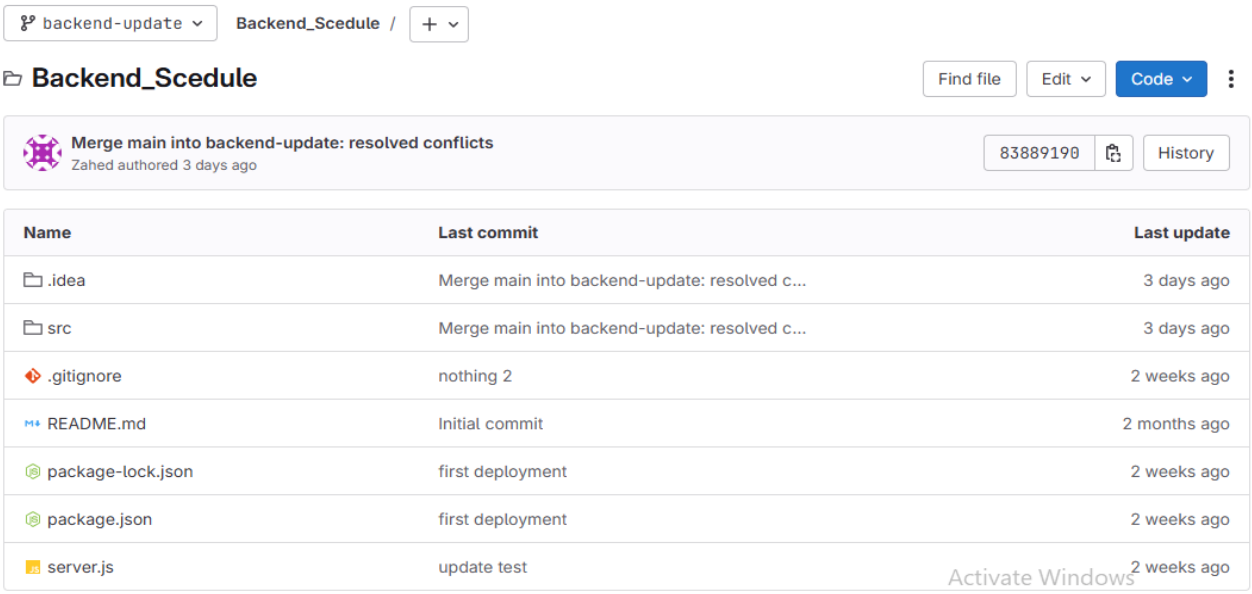


Figure 0-3: : Snapshot Commit History Backend

Similarly, updates to the frontend were committed to the frontend-update branch:

```
git checkout frontend-update
git add .
git commit -m "second deployment"
git push origin frontend-update
```

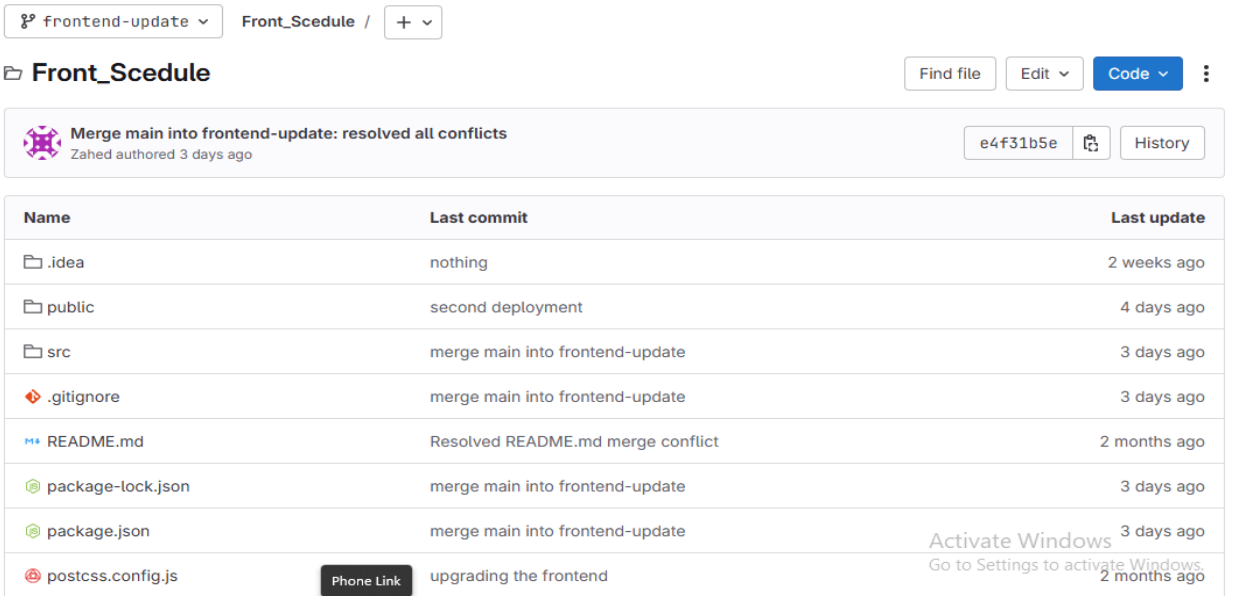


Figure 0-4: Snapshot Commit History Frontend

The following screenshots were taken **after merging the frontend-update and backend-update branches into the main branch**, which is a crucial

step to ensure that only stable, reviewed code is included in the final version of the project.

This workflow allowed for **clean separation of concerns**, **organized tracking of progress**, and easy communication with the project supervisor. Code was reviewed and tested before merging into the main branch to ensure stability and functionality. GitLab also provided a platform for maintaining commit history, enabling version rollback when necessary.



Figure 0-5: General Git Workflow

7.3 Gantt Chart and TimeLine:

The following Gantt chart outlines the full timeline of the project, spanning from April to July 2025. It visually represents the planning and execution phases of the system, including frontend development, backend implementation (divided into major components such as login system, admin panel, student and teacher interfaces), database design, and testing.
Tasks were planned and executed weekly. Testing was conducted in parallel with development to ensure continuous validation of features. Toward the end of the project, focused time was allocated for final bug fixes, report writing, and PowerPoint preparation for the presentation.
This timeline reflects the Agile-inspired workflow adopted throughout the project, with weekly deliverables and progress reviews conducted regularly with the supervisor.

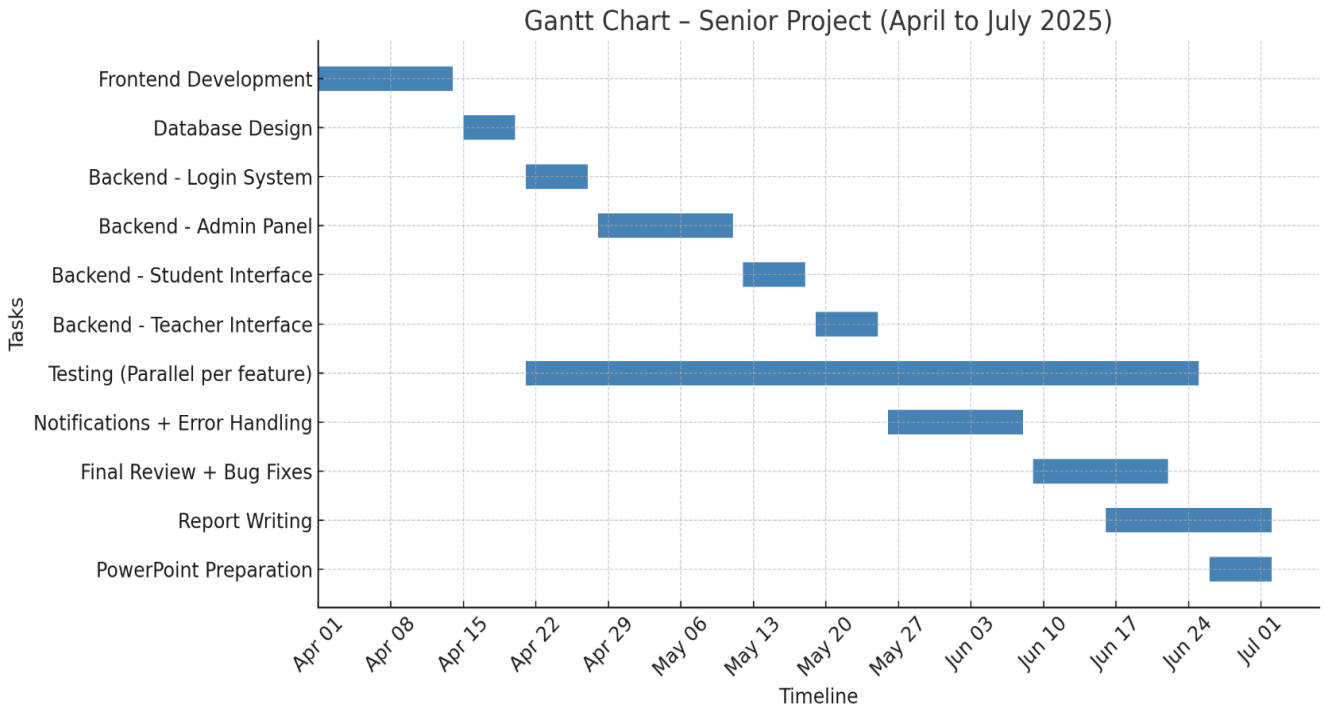


Figure 0-6: Gantt Chart

Chapter 8
Testing and Debugging

8.1 Testing Approach:

The testing approach focused on backend API functionality and application behavior during valid and invalid input scenarios. Testing was performed manually using **Thunder Client** inside **Visual Studio Code** and supported by inspecting database results via **pgAdmin** and runtime debugging using console.log() statements.

The objective was to verify that all API endpoints work as expected, handle errors gracefully, and interact correctly with the PostgreSQL database.

\API Testing with Thunder Client

All API routes were tested using Thunder Client. Requests were sent with:

- Correct data to verify successful operations (e.g., login, add to cart, checkout).
- Missing or incorrect data to validate error handling and server-side validation.

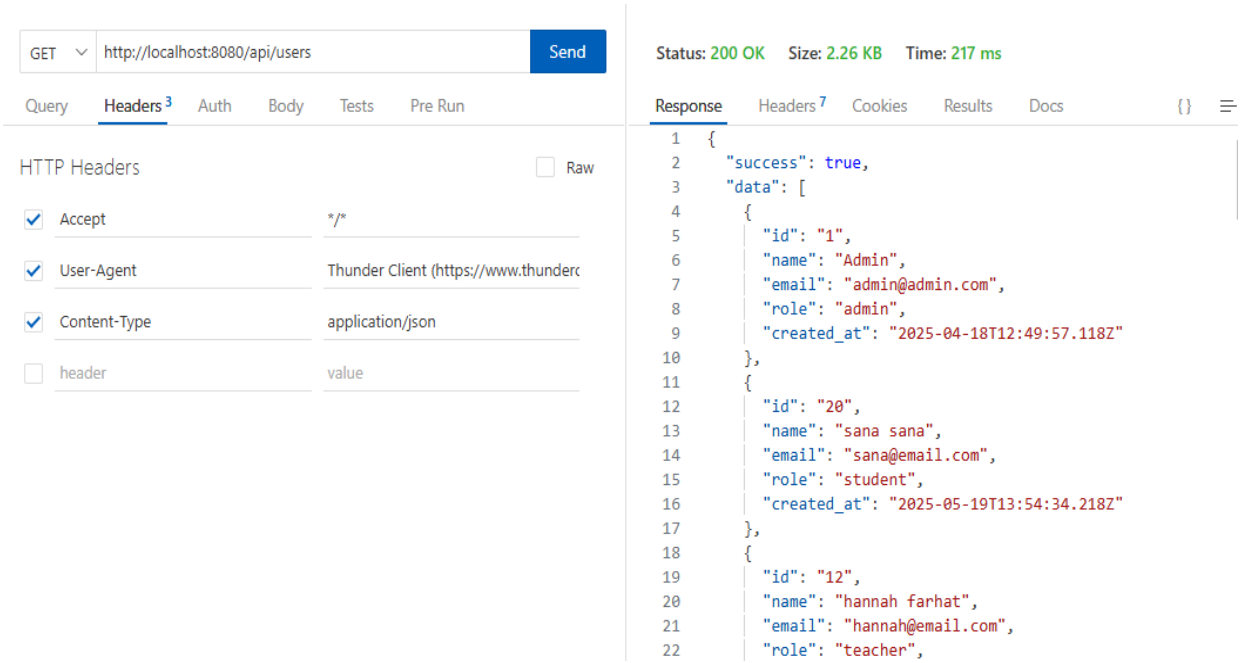


Figure 0-1: Http Request Test (Get)

Database Verification (pgAdmin)

After key operations (like registration, cart updates, or order submission), pgAdmin was used to confirm that the expected changes occurred in the database tables.

public.users/university_timetable/postgres@PostgreSQL 17

No limit

Query | Query History | Scratch Pad

```
SELECT * FROM public.users
ORDER BY id ASC
```

Data Output | Messages | Notifications

Showing rows: 1 to 19 | Page No:

	id [PK] bigint	name character varying (255)	email character varying (255)	password character varying (255)	role character varying (255)
1	1	Admin	admin@admin.com	$2a$10$...	admin
2	2	Professor Smith j	smith@university.com	$2a$10$...	teacher
3	3	john	john@example.com	mypassword123	student
4	11	ahmad ramadan	ahmad.ramadan2005f@gmail.com	$2b$10$2b0rteRbkDgWxFtRKKk0XOzDSyocoZ9z0d7KZNpcsufRuv.GC...	student
5	12	hannah farhat	hannah@email.com	$2b$10$zG/OBwfgAo2UNuhZjxuwRuea5Tt5IEt4RHIu/F7WmsHwe/uP...	teacher
6	18	Ali Khoury	ali@example.com	mypassword123	student
7	20	sana sana	sana@email.com	$2b$10$zlh5bRrGQ6PJS38.lVsv1.ITe.fOFSZA808gTHNQWxm.cs0xEkDiy	student

Figure 0-2: Database Framework Usage

Debugging with console.log()

The console.log() function was used extensively in both the backend and frontend to trace data flow, identify issues, and verify function outputs during development.
Here is an example using this type of testing :
A user is logging in with an incorrect password.

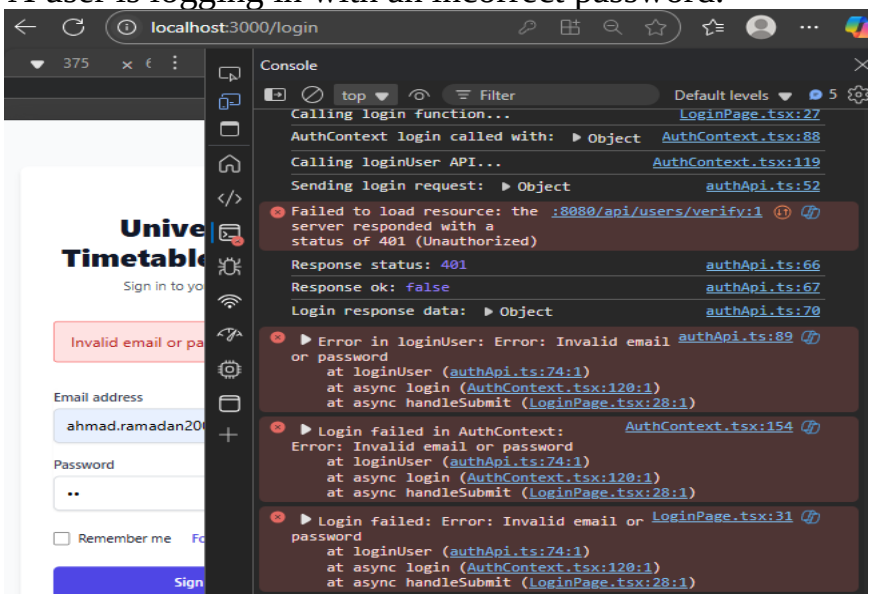


Figure 0-3: Console Debugging

Chapter 9
Data Injection into the Database

9.1 Overview:

To facilitate efficient database population, the project incorporated a data injection mechanism using Python scripts and structured CSV files. This approach streamlined the process of inserting a large volume of consistent and structured data into the system's PostgreSQL database, which was essential for testing and validating the system's functionality.



Figure 0-1: CSV injection

9.2 Tools Used:

- **Python**: The scripting language used for reading CSV files and executing SQL insertions into the database using libraries like psycopg2.
- **VS Code**: The integrated development environment used to write, test, and run the Python scripts.
- **CSV Files**: These files served as the data source. They were manually created or exported, containing structured information about users, class groups, and student enrollments.



Figure 0-2: Python + CSV

9.3 Injected Data Tables:

The following core tables were injected into the database:

- **Users Table**:
  - Data injected from: users_admin.csv, users_teacher.csv, and users_student_injection.py.
  - Each script handled role-specific fields and roles during injection (admin, teacher, student).
- **Student Enrollments Table**:
  - Data injected from: students_and_enrollments.csv via users_student_injection.py.
  - This table mapped students to their respective class groups.
- **Class Groups Table**:
  - Data injected from: class_groups.csv via groups_injection.py.
  - It defined the academic groupings or classes available in the system.

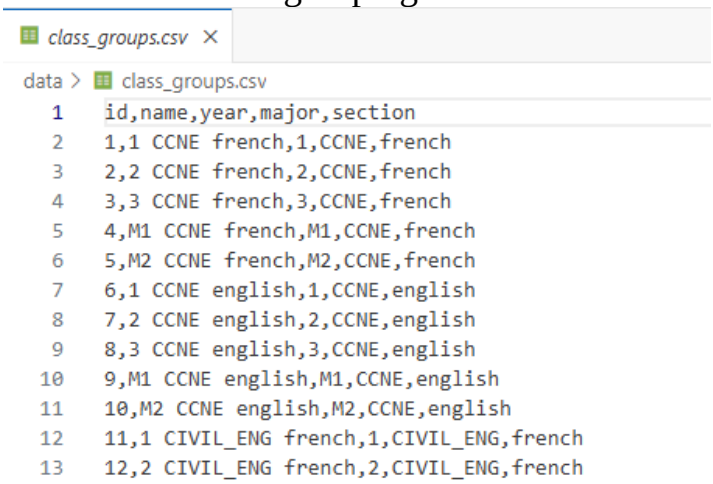


Figure 0-0-3: A sample of inserted data (.CSV file)

9.4 Data Injection Scripts:

The Python scripts were modularized for clarity and reusability:

- users_admin_injection.py, users_teacher_injection.py, and users_student_injection.py handled role-specific user data.
- groups_injection.py injected class group definitions.
- Each script established a connection to the PostgreSQL database using a .env configuration file for secure credential handling.
- Error handling and data validation (e.g., checking for duplicate entries or missing fields) were implemented within the scripts to ensure robust data processing.

9.5 Data Verification:

After injection, additional scripts were used for verification:

- **Verification Scripts**: check_data_users.py, check_data_groups.py, and check_data_enrollement.py.
- **Purpose**: These scripts cross-verified the data in the database against the original CSV files and generated summary reports.
- **Output**: Summary CSVs such as users_check.csv, student_enrollements.csv, and class_groups_check.csv were generated for manual review and confirmation.

9.6 Benefits of Custom Injection:

- **Speed**: Bulk data population without manual entry.
- **Consistency**: Ensures referential integrity across tables (e.g., students linked to valid class groups).
- **Reusability**: Scripts can be reused for future data updates or during testing stages.

Chapter 10
Future Work and Recommendations

10.1 Features to Add in Future:

As the initial version of the scheduling platform reached a stable and functional state, several enhancements and long-term improvements have been identified to increase usability, performance, and scalability.

Future Features and Improvements

- **Drag-and-Drop Calendar Management**
  Implementing an interactive drag-and-drop calendar for the admin would simplify schedule adjustments, making the system more intuitive and reducing manual input errors.
- **Attendance Tracking Module**
  Integrating attendance tracking for students would make the platform even more valuable for both teachers and administrators, supporting features such as absence logs, automatic reporting, and student performance tracking.
- **Multilingual Support (English / French / Arabic)**
  To make the platform accessible to a broader user base in Lebanon, support for English, French, and Arabic will be implemented. Users will be able to choose their preferred language at login or in the settings, and the interface will adjust accordingly. This supports inclusivity across all faculties and user roles.
- **“Remember Me” Functionality with Expiration**
  A “Remember Me” checkbox on the login page will be introduced to store login sessions using secure cookies or localStorage with an expiration date. This improves convenience, especially for students and teachers who access the site frequently.

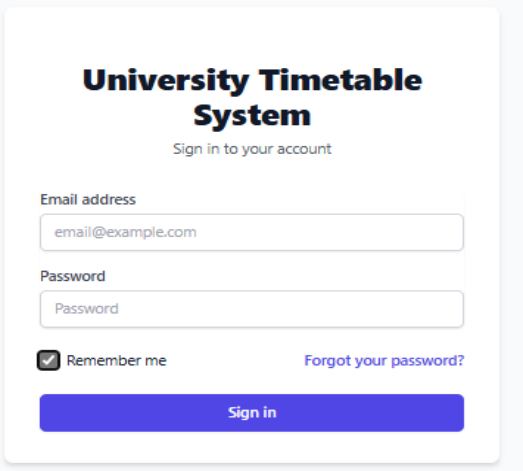


Figure 0-1: Remember-me Feature Page

- **Real-Time Notifications with WebSockets**
  To provide faster communication, the current notification system can be enhanced using WebSocket technology. This will allow instant updates to appear without needing to refresh the page.
- **Admin Analytics Dashboard**
  Adding charts and usage statistics (e.g., login frequency) will help administrators track platform engagement and identify operational bottlenecks.
- **Keycloak or OAuth2 Integration**
  For better scalability and enterprise-grade security, integrating **Keycloak** or another identity management solution would allow centralized login management, token expiration handling, and integration with university identity providers.

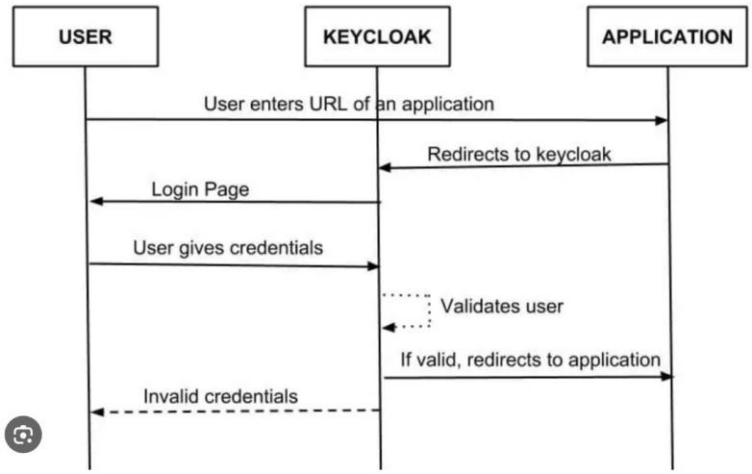


Figure 0-2: KeyCloak Logic

- **Enhanced UI Animations**
  Minor UI transitions and feedback animations (e.g., loading indicators, confirmation toasts) would enhance the user experience and modernize the interface.
- **Dark Mode / Theme Customization**
  To improve accessibility and reduce eye strain during extended use, especially in low-light environments, a dark mode option should be provided for all users. Allowing users to switch between light and dark themes (or enabling automatic theme switching based on system preferences) will enhance comfort and inclusivity. Implementing a dark mode can also modernize the platform's visual appeal and align with current UI/UX best practices.

Scalability and Maintenance Recommendations

- **Use Environment Variables**
  To better manage deployment configurations (API keys, database URLs, etc.), the project should rely on .env files or deployment-time environment variables.
- **Modularize Backend Controllers and Routes**
  As the backend grows, separating logic into modules or service layers will ensure maintainability, code clarity, and testability.
- **Optimize Queries with PostgreSQL Views**
  Using PostgreSQL views for frequently joined queries (like schedule lookups) can improve performance and simplify backend logic.

Conclusion

This project successfully culminated in the development of a robust, full-stack web application designed to streamline the management and consultation of academic schedules at the Faculty of Technology – Lebanese University, Saida Branch. By addressing critical challenges such as lack of real-time schedule updates, poor communication of changes, and the absence of a centralized digital system, the platform delivers significant improvements in both efficiency and user experience.

Through a carefully designed interface, students and instructors are now able to access personalized schedules, receive instant notifications, and interact with a system tailored to the specific structure of Lebanese public institutions. Administrators benefit from a user-friendly dashboard that simplifies complex tasks such as managing courses, assigning instructors, and adjusting schedules.

Beyond its current implementation, the project lays a solid foundation for future expansion, including features such as multilingual support, attendance tracking, mobile responsiveness, and real-time data synchronization. It demonstrates the practical application of web technologies—such as React, Node.js, and PostgreSQL—in solving real-world problems and highlights the importance of user-centered design and system scalability in modern software engineering.

Ultimately, this work contributes to the ongoing digital transformation of education in Lebanon and serves as a strong example of how targeted, context-aware solutions can improve institutional workflows and enhance academic communication.

References

- React Official Docs: https://react.dev/
- TypeScript Handbook: https://www.typescriptlang.org/docs/
- Express.js: https://expressjs.com/
- PostgreSQL Docs: https://www.postgresql.org/docs/
- JWT Auth Guide: https://jwt.io/introduction
- React Hook Form: https://react-hook-form.com/
- PostgreSQL Enum Types: https://www.postgresql.org/docs/current/datatype-enum.html
- GitHub Documentation : https://docs.github.com/